

Revisiting Time-Space Tradeoffs in Collision Search and Decision Problems

Jian Guo, Wenjie Nan^(✉), and Yiran Yao

Nanyang Technological University, Singapore
guojian@ntu.edu.sg, wenjie006@e.ntu.edu.sg, yiran005@e.ntu.edu.sg

Abstract. We present analysis of time-space tradeoffs for both the search and decision variants of the k -collision problem in algorithmic perspective, where $k \in [2, O(\text{polylog}(N))]$ and the underlying function is $f_{N,M} : [N] \rightarrow [M]$ with $M \geq N$. In contrast to prior work that focuses either on 2-collisions or on random functions with $M = N$, our results apply to both random and arbitrary functions and extend to a broader range of k . The tradeoffs are derived from explicit algorithmic constructions developed in this work, especially for decision problems when $k \geq 3$.

For 2-collision problems, we show that for any random function $f_{N,M}$ with $M \geq N$, the time-space tradeoff for finding all 2-collisions follows a single curve $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$, where T denotes time complexity and S denotes available space. This tradeoff also extends to arbitrary functions with at most $O(N)$ total 2-collisions.

For 3-collision problems, we identify two time-space tradeoff curves for the search variant over random functions, depending on the available space S . For arbitrary functions, we show that the decision problem can be solved with a tradeoff of $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}} + \frac{N}{S} \frac{n_2}{n_3}\right)$, where n_i denotes the number of i -collisions. Surprisingly, for random functions, the decision problem for 3-collision shares the same time-space tradeoff as the 2-collision case $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$.

For general k -collision problems, we extend these results to show that the decision problem over arbitrary functions can be solved in time $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}} + \frac{N}{S} \frac{n_2}{n_k}\right)$. For the search problem over random functions, we derive two time-space tradeoffs based on the space S , yielding approximately $S^{1/(k-2)}$ or $S^{1/(2k-2)}$ -fold speedups compared to the low-memory setting $S = O(\log M)$. When $M = N$, the tradeoff simplifies to one single curve with $S^{1/(k-2)}$ -fold speedup.

Keywords: k -Collision · Time-space tradeoff

1 Introduction

Collision problems are fundamental in cryptography, from both theoretical and practical perspectives. Over the past few decades, many beautiful results have been made in this area, including the development of efficient algorithms for finding 2-collisions [33, 32, 10] and k -collisions [26, 31, 27, 22] in both classical and quantum settings, as well as analyses of their optimality in terms of time

complexity [28, 1] and time-space tradeoffs [37, 11, 17, 38]. Previous researches primarily focused on search problems for random functions $f_{N,N} : [N] \rightarrow [N]$, where $[N] = \{0, 1, \dots, N-1\}$. More recently, attention has expanded to low-memory algorithms [39, 13] and time-space tradeoffs for both search and decision problems on 2-collisions concerning more general functions $f_{N,M} : [N] \rightarrow [M]$ [10], and other related problems such as *list distinctness* and *random k -sum* [4].

In this paper, we investigate the k -collision problem for functions $f_{N,M} : [N] \rightarrow [M]$, where $2 \leq k \leq \text{polylog}(N)$ and $M \geq N$. A k -collision is a set of k distinct inputs $(x_1, x_2, \dots, x_k)$ such that $f(x_i) = f(x_j)$ for all $i, j \in [1, k]$. We consider both the *search* and *decision* versions of the problem: the former asks for one or several k -collision instances given access to f , while the latter asks whether at least one k -collision exists.

For the search problem, we study random functions $f_{N,M} : [N] \rightarrow [M]$ and extend the analysis to arbitrary functions. A natural question arises when $M \geq N$: since we can reduce the problem of finding k -collisions in a random function $f_{M,M}$ to the same problem in $f_{N,M}$ for $M \geq N$, does this make $f_{N,M}$ strictly harder in terms of time-space tradeoffs? Although we do not establish a formal separation, we show that the best known algorithm for $f_{N,M}$ has a worse time-space tradeoff than for $f_{M,M}$. This generalizes Joux & Lucks's result [27] for $f_{N,N}$, which achieves optimal time $T = \tilde{\Theta}(N^{(k-1)/k})$ with space $S = \tilde{\Theta}(N^{(k-2)/k})$, but does not provide matching tradeoffs when $S \ll N^{(k-2)/k}$.

Another central question is the time-space tradeoff for finding *all* 2-collisions in $f_{N,M}$, particularly for random functions. This is motivated by applications such as golden collisions and the k -collision decision problems for $k \geq 3$. We show that for any random function $f_{N,M}$ with $M \in [N, N^2]$, the time-space tradeoff for finding all 2-collisions depends only on N . Specifically, for functions with at most $O(N)$ 2-collisions—which is typical for random functions—the time-space tradeoff remains the same, independent of the exact number of 2-collisions.

Consequently, our results suggest that for random functions $f_{N,M}$ with $M \geq N$, searching for a *fixed number* of 2-collisions in $f_{N,M}$ is at least as hard as doing so in $f_{M,M}$. However, for computing *all* 2-collisions, random $f_{N,M}$ is asymptotically as hard as random $f_{N,N}$.

For the decision problem—also known as *k -element distinctness*—extensive work exists in the quantum setting on minimizing query complexity [2, 8, 7, 30, 25], but classical results remain limited, especially regarding time-space tradeoffs beyond $k = 2$. When $k = 2$, Beame et al. [5] showed that with access to a random oracle, one can solve 2-element distinctness using a randomized Monte Carlo algorithm with time-space tradeoff $T = \tilde{O}(N^{3/2}/\sqrt{S})$. Chen et al. [14] subsequently removed the random oracle assumption via pseudorandomness when memory is small, $S \sim \log M$, and Lyu & Zhu [29] further generalize the idea to obtain a complete tradeoff curve for general memory without random oracle.

However, several questions remain for k -element distinctness. First, given the 2-element tradeoff, how much harder is the general k -element case? For example, if a function has only $\binom{k}{2}$ 2-collisions and *one* k -collision, the problem is roughly as hard as the 2-element case: once we find one 2-collision, we can potentially

recover the k -collision with $O(N)$ time and $\tilde{O}(1)$ space. In contrast, if the function has $O(N)$ 2-collisions but only one k -collision, it may be easy to find a 2-collision, yet still very difficult to determine whether a k -collision exists. This suggests a potential connection between the number of 2-collisions and the hardness of the k -element distinctness problem. Can this relationship be formalized in terms of time-space tradeoffs? Furthermore, are there non-trivial instances—e.g., random functions—where the k -element distinctness problem becomes easier?

For random functions, most values of $M \geq N$ lead to trivial decision problems with overwhelming probability. For instance, in the 3-collision case, if $M = O(N^{1.5-\varepsilon})$, the answer is almost surely “yes”; if $M = \Omega(N^{1.5+\varepsilon})$, it is almost surely “no”. Therefore, we focus on the *non-trivial* regime where $M = \tilde{\Theta}(N^{k/(k-1)})$, in which both outcomes occur with non-negligible probability. Other non-trivial cases include those where $k \sim \frac{\log N}{\log \log M}$. For these regimes, we ask: how difficult is it to decide the existence of a k -collision, and how does this difficulty depend on M ?

In this work, we study these questions from an algorithmic perspective by presenting explicit algorithms with provable time-space tradeoffs for the upper bound, and by discussing conditional lower bounds. Our results are not only of theoretical interest but also relevant to recent studies on the security of symmetric-key primitives against space-bounded adversaries [36, 20, 15, 23].

In particular, we show that for random functions, non-trivial decision problems become easier in terms of time-space tradeoffs; for example, the decision problems for 2-collision and 3-collision share the same tradeoff. For arbitrary functions $f_{N,M}$ with $M \geq N$, we identify two key parameters that determine the tradeoff: the number of 2-collisions n_2 and the number of k -collisions n_k , for small $k = O(\text{polylog}(N))$. In fact, the tradeoff is governed by the ratio $\frac{n_2}{n_k}$.

1.1 Our Results

In this work, we provide a comprehensive analysis of the time-space tradeoffs for both the search and decision variants of the k -collision problem, for any small $k = O(\text{polylog}(N))$. Our results are divided into two categories: random functions and arbitrary functions. We consider functions of the form $f_{N,M} : [N] \rightarrow [M]$, where $M \geq N$. When $M < N$, the domain size can be reduced to $N' = M$, allowing our results to be applied directly.

The table 1 summarizes the time-space tradeoffs we obtain, where n_k denotes the number of k -collisions of the function in the case of arbitrary functions $f_{N,M}$. We assume the parameter $\lambda = O(\log N)$ and absorb such terms (including $\log M$) into the $\tilde{O}$ notation. We highlight several key findings:

1) Searching for C_2 many 2-collisions over a random function $f_{N,M}$ has time-space tradeoff $T = \tilde{O}\left(\frac{C_2 M}{\sqrt{NS}}\right)$, provided $C_2 \gtrsim \frac{SN}{M}$. This is worse than the tradeoff for a random function $f_{M,M}$, which achieves $T = \tilde{O}\left(\frac{C_2 \sqrt{M}}{\sqrt{S}}\right)$ with $C_2 \gtrsim S$.

Table 1. Time-Space Tradeoffs for Collision Problems with Probability $1 - O(\text{negl}(\lambda))$. Gray ones represent previous results. N and M are domain size and codomain size of $f_{N,M}$. S is the space budget. λ is a parameter related to the success rate, where $1 - O(\text{negl}(\lambda))$ denotes the algorithm's success rate. n_k is total number of k -collisions. C_k is target number of k -collisions.

Scenario	Problem Type	Function Type	Time-Space Tradeoff
$k = 2$	Search for C_2 2-collisions	Random	$O\left(\lambda \log^{\frac{1}{2}}(M) \cdot \frac{C_2 M}{\sqrt{NS}}\right)$
		Arbitrary	$\begin{cases} O\left(\lambda \log^{\frac{1}{2}}(M) \frac{C_2 N^{\frac{3}{2}}}{n_2 \sqrt{S}}\right) & \text{if } n_2 \leq N \\ O\left(\lambda \log^{\frac{1}{2}}(M) \frac{C_2 N}{\sqrt{n_2 S}}\right) & \text{if } n_2 > N \end{cases}$
	Search for all 2-collisions	Random	$O\left(\lambda \log^{\frac{3}{2}}(M) \frac{N^{\frac{3}{2}}}{\sqrt{S}}\right)$
		Arbitrary	$\begin{cases} O\left(\lambda \log^{\frac{3}{2}}(M) \frac{N^{\frac{3}{2}}}{\sqrt{S}}\right) & \text{if } n_2 \leq N \\ O\left(\lambda \log^{\frac{3}{2}}(M) \frac{N \sqrt{n_2}}{\sqrt{S}}\right) & \text{if } n_2 > N \end{cases}$
	Decision	Random	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}}\right), M = \Theta(N^2)$ [5]
		Arbitrary	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}}\right)$ [5]
$k = 3$	Search for C_3 3-collisions	Random	$\begin{cases} \tilde{O}\left(\frac{C_3 M}{S}\right), S \leq \tilde{\Theta}\left(C_3^{\frac{2}{3}} N^{\frac{1}{3}}\right) \\ \tilde{O}\left(\frac{C_3^{1/2} M}{(NS)^{\frac{1}{4}}}\right), S \in \left[C_3^{\frac{2}{3}} N^{\frac{1}{3}}, C_3^{\frac{2}{3}} \frac{M^{\frac{4}{3}}}{N}\right] \end{cases}$
		Arbitrary	$\tilde{O}\left(\frac{C_3 N}{S} \frac{n_2}{n_3}\right), S \leq \begin{cases} \tilde{\Theta}\left(\left(\frac{C_3 n_2}{n_3}\right)^{\frac{2}{3}} N^{-\frac{1}{3}}\right) & \text{if } n_2 \leq N \\ \tilde{\Theta}\left(\frac{C_3^{\frac{2}{3}} n_2}{n_3^{\frac{2}{3}}}\right) & \text{if } n_2 > N \end{cases}$
	Decision	Random	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}}\right), M = \tilde{\Theta}\left(N^{\frac{3}{2}}\right)$
		Arbitrary	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}} + \frac{N n_2}{S n_3}\right)$
General k	Search for C_k k -collisions	Random	$\begin{cases} \tilde{O}\left(C_k^{\frac{1}{k-2}} \frac{M}{S^{\frac{1}{k-2}}}\right), S \leq \tilde{O}\left(C_k^{\frac{2}{k}} N^{\frac{k-2}{k}}\right) \\ \tilde{O}\left(\frac{C_k^{\frac{1}{k-1}} M}{(NS)^{\frac{1}{2(k-1)}}}\right), S \in \left[C_k^{\frac{2}{k}} N^{\frac{k-2}{k}}, \frac{C_k^{\frac{2}{k}} M^{\frac{2k-2}{k}}}{N}\right] \end{cases}$
		Arbitrary	$\tilde{O}\left(\sum_{t=2}^{k-1} \frac{C_{t+1}}{C_t} \frac{n_{c,t} N}{n_{c,t+1}}\right) + \tilde{O}\left(\frac{M \sqrt{S}}{\sqrt{N}}\right)$
	Decision	Random	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}} + \frac{N^{\frac{2k-3}{k-1}}}{S}\right), M = \tilde{\Theta}\left(N^{\frac{k}{k-1}}\right)$
		Arbitrary	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}} + \frac{N^3}{MS}\right), k = \Theta\left(\frac{\log N}{\log \log M}\right)$
		Arbitrary	$\tilde{O}\left(\frac{N^{\frac{3}{2}}}{\sqrt{S}} + \frac{N n_2}{S n_k}\right)$

2) Searching for *all* 2-collisions has the same time-space tradeoff $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$ for any function $f_{N,M}$, as long as the number of 2-collisions satisfies $n_2 \leq O(N)$. This bound holds in particular for random functions $f_{N,M}$ with $M \geq N$.

3) For search problem of k -collision over random functions, we have

$$T = \begin{cases} \tilde{O}\left(C_k^{\frac{1}{k-2}} \frac{M}{S^{\frac{1}{k-2}}}\right), & S \leq \tilde{O}\left(C_k^{\frac{2}{k}} N^{\frac{k-2}{k}}\right) \\ \tilde{O}\left(\frac{C_k^{\frac{1}{k-1}} M}{(NS)^{\frac{1}{2(k-1)}}}\right), & S \in \left[C_k^{\frac{2}{k}} N^{\frac{k-2}{k}}, \frac{C_k^{\frac{2}{k}} M^{\frac{2k-2}{k}}}{N}\right]. \end{cases}$$

This extends the result of Joux & Lucks [27] on $f_{N,N}$, which only provides the optimal time complexity and the corresponding memory, to the full range of memory sizes. In addition, when $k \geq 3$, the search problem exhibits two distinct tradeoff curves, as shown above. See Figure 1 for an illustration in the case $k = 3$. Notably, only the second (and asymptotically worse) curve is applied to the 2-collision search problem, regardless of the available memory S .

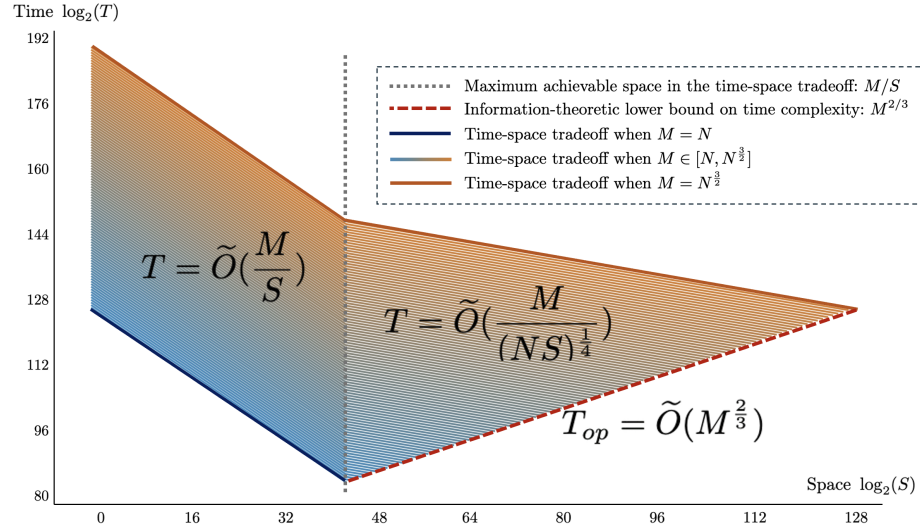


Fig. 1. Time-space tradeoff for searching *one* 3-collision ($C_3 = 1$) of random functions $f_{N,M}$, giving $M \in [N, N^{3/2}]$, $N = 2^{128}$.

4) The decision version of the 3-collision problem for random functions $f_{N,M}$ has the same time-space tradeoff as the 2-collision decision problem, namely $\tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$, except with negligible probability of error.

5) For arbitrary functions, the time-space tradeoff for the decision k -collision problem depends on the ratio $\frac{n_2}{n_k}$ (for small k), suggesting the hardest case may arise when $n_2 = O(N)$ and $n_k = O(1)$. When $n_2 \gg N$, the problem becomes trivial due to the pigeonhole principle. Moreover, distinguishing whether $n_2 \gg N$ can be done in time at most $\tilde{O}(\sqrt{N})$ using only $O(\log M)$ memory.

Remark 1. The multi-chain algorithms used in this paper can be naturally extended to a parallel setting using the distinguished point method [32], which is closely related to the multi-chain approach inspired by Hellman’s method [21]. The same analysis applies, as the number of distinct points (key statistic in the paper) over chains remains asymptotically unchanged under a suitable condition for identifying distinguished points, see Section 2,3 for details.

Remark 2. Our analysis is not only of theoretical interest, and it is also relevant to recent studies on the security of symmetric-key primitives against space-bounded adversaries [36, 20, 15, 23].

1.2 Related Works

Applications of chained algorithms in collision problems. The Floyd’s cycle-finding algorithm was applied on 2-collision search [33], then used by Beame et al. [5] to design a randomized algorithm for the 2-element distinctness problem, achieving a time-space tradeoff of $T = \tilde{O}(N^{3/2}/\sqrt{S})$. Multi-chain variants of the algorithm have been employed in parallel/multiple 2-collision search [32] as well as in the 2-element distinctness problem [5, 29]. Related techniques have also been used to analyze the list distinctness problem, where the number of collisions within each list serves as a central parameter [4].

Dinur et al. [18] proposed a recursive algorithm to identify large components in a random function $f_{N,N}$, which corresponds to a k -collision when k is large, $k = \Omega(N^c)$. Unlike other works, which discard everything after each Floyd cycle, their method feeds inner cycle outputs into outer cycles, boosting the probability of detecting large components. However, for small components of size $O(\text{polylog}(N))$, this strategy offers no asymptotic advantage over simpler one-pass method.

Statistical Measures in Collision Problems. Flajolet and Odlyzko [19] introduced many useful statistical measures for analyzing random functions $f_{N,M}$, focusing on $M = N$. Bellare and Kohno [6] proposed the notion of *balance*, defined for a function $f_{N,M} : [N] \rightarrow [M]$ as $\mu_2(f_{N,M}) = \log \left(\frac{N^2}{\sum_{i=1}^M P_i^2} \right)$, where P_i is the number of preimages of output $i \in [M]$. This captures the function’s regularity and relates to collision resistance. Ramanna and Sarkar [34] generalized this to higher-order collisions by defining $\mu_k(f_{N,M}) = \log \left(\frac{N^k}{\sum_{i=1}^M P_i^k} \right)$.

In this work, we focus on a simpler but effective statistic: n_k , the number of k -collisions in the function. In particular, n_2 plays a central role in our analysis, especially for decision problems, where $\mu_2(f_{N,M}) = \log \left(\frac{N^2}{2n_2 + N} \right)$.

k -Collision and Related problems. For the case of $k = 2^t$, Joux [26] proposed an algorithm for chained hash functions with time complexity $\tilde{O}(t\sqrt{M})$ and Nandi & Stinson extended the attack to more generalized sequential hash functions [31]. Later on [27], they presented an algorithm for finding 3-collisions in random functions $f_{N,N}$, achieving a time-space tradeoff of $\tilde{O}(N/S)$. They further showed an algorithm achieving the optimal time complexity $T = \tilde{O}(N^{(k-1)/k})$ for finding a single k -collision using space $S = \tilde{O}(N^{(k-2)/k})$ for general k .

In the quantum setting, Hosoyamada et al. [22] introduced a quantum algorithm for searching k -collisions, which was later improved by Liu and Zhandry [28], where they also established query lower bounds. In particular, Bonnetain et al. [10] showed how to use quantum random walk to search for many 2-collisions for random functions $f_{N,M}$, $M \in [N, N^2]$. In addition, several works [2, 8, 7, 30, 25] based on quantum walks focus on the decision variant, *k-element distinctness*, aiming for minimizing the number of queries in quantum setting.

Recently, researchers also investigated the complexity-theoretic aspects of k -collision problems, uncovering connections to other domains in cryptographic and computational complexity [24, 9].

Another important line of research concerns the study of k -collision resistance [35, 12]. While this assumption is formally weaker than standard collision resistance, Buzek and Tessaro [12] showed that infinitely-often secure collision-resistant hash functions can be constructed from infinitely-often secure k -collision-resistant hash functions, where infinitely-often secure means the adversary fails for infinitely many security parameters n .

1.3 Roadmaps

We begin with an overview of our techniques in Section 2, where we introduce the core ideas underlying our time-space tradeoffs. Section 3 presents preliminaries and key statistical measures used throughout the paper. In Sections 4, 5, and 6, we analyze the 2-collision, 3-collision, and general k -collision problems in detail.

2 Technical Overview

To begin, we briefly introduce the 2-element distinctness algorithms by Beame et al. [5] (BCM) and a modified version for time-space tradeoffs by Lyu & Zhu [29]. The problem asks whether a function $f_{N,M} : [N] \rightarrow [M]$ contains any pair (x_i, x_j) with $f(x_i) = f(x_j)$ and $x_i \neq x_j$.

BCM algorithm. Given a random hash function $H : [M] \rightarrow [N]$, define a directed graph $G_{f_{N,M}, H}$ where vertices correspond to elements in domain $[N]$ of $f_{N,M}$, with an edge from x_i to x_j if and only if $H(f_{N,M}(x_i)) = x_j$. Define $G_{f_{N,M}, H}(x_i)$ and $G_{A, H}(\{x_{i_1}, \dots, x_{i_s}\})$ as the subgraphs reachable from x_i , or any of vertices in set $A = \{x_{i_1}, \dots, x_{i_s}\}$. Each $G_{f_{N,M}, H}(x_i)$ forms a chain starting at

x_i , while $G_{f_{N,M},H}(\{x_{i_1}, \dots, x_{i_s}\})$ consists of multiple such chains. In particular, the chain beginning at $x_1 \in [N]$ develops as:

$$[x_1, y_1 = f_{N,M}(x_1)] \rightarrow \dots \rightarrow [x_i = H(y_{i-1}), y_i = f_{N,M}(x_i)] \rightarrow \dots$$

Consider a colliding pair $f_{N,M}(x_{i^*}) = f_{N,M}(x_{j^*})$ ($i^* \neq j^*$) in $f_{N,M}$. For any randomly chosen H and starting point x_i , the probability that x_{i^*} appears in $G_{f_{N,M},H}(x_i)$ is $\Omega(1/\sqrt{N})$, with the same probability holding for x_{j^*} , as long as the number of 2-collision of $f_{N,M}$ is less than $O(N)$. This follows because the subgraph (or equivalently, the chain) contains $\Theta(\sqrt{N})$ distinct random points in expectancy. Consequently, the probability that both colliding elements appear on the same chain is $\Omega(1/N)$. By repeating for $O(N)$ times, we can ensure detection of the collision pair x_{i^*}, x_{j^*} with overall time complexity $O(N^{3/2})$ using space $O(\log N)$.

For time-space tradeoffs with $O(S \log N)$ space, BCM introduced an algorithm that identifies all collisions within the subgraph $G_{f_{N,M},H}(\{x_{i_1}, \dots, x_{i_S}\})$ with time complexity $\tilde{O}(|G_{f_{N,M},H}(\{x_{i_1}, \dots, x_{i_S}\})|)$. The main idea of the algorithm is to reduce the size of a cycle by half each time it is traversed, so preventing the algorithm from repeatedly visiting the same segments of the graph.

Lyu & Zhu's work. Lyu & Zhu use a different multi-chain approach similar to the technique proposed by Hellman [21] and parallel collision search [32]. Their method generates S chains using space $O(S \log N)$ from multiple starting points $x_{i_1}, \dots, x_{i_S}$ and stop each chain after approximately $\sqrt{N/S}$ steps. Furthermore, they show that any collision pair of $f_{N,M}$ (if one exists) will be contained within these chains with probability $\Omega(S/N)$. This yields identical time-space tradeoffs to those established in BCM's original work. The fundamental reason both approaches - BCM's graph search algorithm and the simpler shorter-chain method - achieve the same tradeoff is that they each effectively examine $O(\sqrt{SN})$ distinct random points across the chains and can detect up to $\sim S$ collisions of $f_{N,M}$ or H , requiring $O(\sqrt{SN})$ time complexity.

Moreover, they demonstrate how to substitute the random oracle with pseudorandomness. We state their main result as follows:

Lemma 1 (2-element distinctness [29]). *Given oracle access to any function $f_{N,M} : [N] \rightarrow [M]$ with one collision pair $(x_1, x_2) \in [N]^2$ where $x_1 \neq x_2$ and $f_{N,M}(x_1) = f_{N,M}(x_2)$, there exists a Monte Carlo algorithm $\mathcal{A}$ that finds (x_1, x_2) with time complexity $\tilde{O}(N^{3/2}/\sqrt{S})$ and space complexity $O(S)$.*

Remark 3. For convenience, we sometimes write $O(S)$ to indicate $O(S)$ chains rather than $O(S)$ bits of space in this technical overview. In our formal analysis, $O(S)$ bits of space corresponds to $O(S/(\log N + \log M))$ chains. Consequently, the time complexity increases by $O(\text{poly}(\log N))$ factors, which are sufficiently small to be absorbed within the $\tilde{O}$ notation.

2.1 Basic Algorithms

We introduce some basic algorithms that will be used throughout the paper.

Chain-based algorithms. We present two chain-based algorithms based on available memory S . For very low memory settings, the single-chain algorithm constructs a chain of length ℓ as follows:

$$\begin{aligned} x_1 &\leftarrow \text{\$}[N], y_1 = f_{M,N}(x_1) \\ x_i &= H_{M,N}(y_{i-1}), y_i = f_{N,M}(x_i), \quad i \in [2, \ell]. \end{aligned}$$

Collisions within the chain are detected using Floyd's cycle-finding algorithm, which requires $O(\ell)$ time complexity and $O(\log M)$ memory.

For settings with memory S which is capable of storing approximately $\frac{S}{2 \log M}$ chains of length ℓ , we use the similar method to construct $\tilde{O}(S)$ independent chains based on independent hash functions $\{H_{M,N}^u\}$ where we omit logarithmic factors for convenience. During chain generation, we implement a collision detection mechanism: for each new chain j , we check for collisions with all previous chains by computing ℓ more points:

1. Compute 2ℓ points $(x_i^j, y_i^j)_{i \in [1, 2\ell]}$;
2. During computation, check whether $\exists j' < j, i \leq 2\ell$ such that $y_i^j = y_{end}^{j'}$.

After checking, the detection requires $O(\ell)$ additional queries to $f_{N,M}$ to verify if it is a collision. Upon detecting a collision, we terminate processing of subsequent points in chain j since they would duplicate points from chain j' . We also check for intra-chain collisions. Since each chain yields at most 2 detectable collisions, the total time complexity for collision finding remains $\tilde{O}(S\ell)$, matching the chain generation complexity. We refer readers to Algorithm 1 for further details.

Multi-target preimage algorithm. If there are S known $(k-1)$ -collisions $\{(x_1^i, \dots, x_{k-1}^i, y^i)\}_{i \in [1, S]}$ of a function $f_{N,M}$, the multi-target preimage algorithm for finding a k -collision proceeds:

1. Randomly select $x \in [N]$ and compute $y = f_{N,M}(x)$;
2. If there exists an index $j \in [1, S]$ such that $y = y^j$ and $x \notin \{x_1^j, \dots, x_{k-1}^j\}$, output the k -collision $(x_1^j, x_2^j, \dots, x_{k-1}^j, x, y^j)$. Otherwise, go back to step 1.

2.2 2-collision problems

For 2-collision problems of some $f_{N,M}, M \geq N$, we typically concern four important problems: (1) find some 2-collisions, (2) find all 2-collisions, (3) estimate the number of 2-collisions, and (4) decide whether any 2-collision exists.

The decision problem (4) has been resolved in previous work on 2-element distinctness, achieving time-space tradeoffs of $\tilde{O}(N^{3/2}/\sqrt{S})$ [5, 29]. In particular, Lyu and Zhu's work demonstrates that given the second frequency moment F_2 of $f_{N,M}$, there exists Monte Carlo algorithms using space S where the probability of detecting any collision pair $x_1, x_2 \in [N]$ is lower bounded by $\tilde{\Omega}(S/F_2(f_{N,M}))$.

Number of distinct points over chains and chain length. In this work, we introduce a direct statistical measure for collision detection: the number of distinct points n_d in a single or multiple chains. Closely related to chain length, n_d serves two main purposes. First, it guides the choice of optimal chain lengths under memory constraints S . Second, it provides a simpler and more intuitive basis for analyzing the collision probability in chain-based algorithms.

In addition, the time complexity to find a single 2-collision can be used to estimate the total number of 2-collisions n_2 , which is also related to the optimal chain length.

When the number of 2-collisions is $n_2 \leq O(N)$, we expect the longest single chain to have length $\sim \sqrt{N}$, since the number of distinct points in a chain

$$(x_i, y_i), \quad \text{where} \quad x_i = H(y_{i-1}), \quad y_i = f_{N,M}(x_i),$$

depends on collisions occurring both on the x_i and the y_i values. The probability of a collision on x_i is approximately $\frac{i-1}{N}$, assuming the previous $i-1$ points on the chain are distinct. Meanwhile, the probability of a collision on y_i is roughly $\frac{i \cdot n_2}{N^2}$, as there are $i-1$ possible pairs (x_j, x_i) for $j < i$, and n_2 total 2-collisions. More precisely, the collision probability on y_i is

$$1 - \left(1 - \frac{n_2}{N^2}\right)^{i-1}.$$

We observe that as long as $n_2 \leq O(N)$, the collision probability is dominated by collisions on x_i . Consequently, the effective maximum chain length is approximately $\sqrt{N}$, and by constructing roughly S chains, each of length $\ell = \sqrt{N/S}$, we expect to traverse about $\sqrt{NS}$ distinct random points in total. If we choose a longer chain length, although we can traverse more distinct points, the overall time complexity $S\ell$ would become much larger than additional number of distinct points obtained, thus reducing the overall efficiency. When $n_2 \gg N$, the collision probability becomes dominated by collisions on y_i , and the effective maximum chain length reduces to approximately $\frac{N}{\sqrt{n_2}} \ll \sqrt{N}$. In this regime, to balance the number of distinct points and time complexity for computing the S chains, each chain should have length about $\frac{N}{\sqrt{n_2 S}}$.

From distinct points to finding 2-collisions. By lower bounding the number of distinct points n_d based on the number of 2-collisions n_2 , we can directly compute the probability of finding each collision pair using either the single-chain or multi-chain method. This is possible because the starting points are chosen randomly and the hash function H is also random. As a result, the distinct points collected across chains are independent and uniformly random. Therefore, for any pair $x_1 \neq x_2$, the probability that both x_1 and x_2 are included among the distinct points is approximately

$$\frac{n_d}{N} \cdot \frac{n_d - 1}{N} \sim \frac{n_d^2}{N^2}.$$

We begin with the case of random functions $f_{N,N}$, which will later be extended to $f_{N,M}$ with $M \geq N$. As n_2 is a random variable, it satisfies $n_2 = O(N)$ with overwhelming probability. This allows chain-based algorithms with space S to collect $\sim \sqrt{NS}$ distinct points. Since any specific 2-collision appears with probability roughly $\frac{n_d^2}{N^2} \sim \frac{NS}{N^2} = \frac{S}{N}$, this algorithm or similarly PCS [32] achieves a $\sqrt{S}$ -fold speedup over running Floyd's algorithm independently on S chains: the success probability scales with S , while the total runtime increases by only $\sqrt{S}$ times. Given $\Omega(N)$ total 2-collisions, this implies that $\Omega(S)$ are expected to be found.

Furthermore, generating S collisions in codomain $[N]$ requires time $\sqrt{NS}$, which is known to be optimal even with additional space [17]. Thus, achieving a $\sqrt{S}$ -fold speedup requires targeting at least S collisions. If the goal is only $S' < S$ 2-collisions, then a $\sqrt{S'}$ -fold speedup is the best possible.

Now we extend to any random function $f_{N,M}$ with $M \geq N$, we can always truncate the output from $[M]$ to $[N]$. This allows us to find $\sim S$ pseudo-collisions in the reduced space $[N]$, which correspond to roughly $S \cdot \frac{N}{M}$ true 2-collisions in the original space $[M]$. This observation enables us to analyze the time-space tradeoffs for finding C_2 2-collisions for random functions $f_{N,M}$ where $M \geq N$.

To find C_2 2-collisions in output space $[M]$ using space S , we can proceed by first finding $C_2 \cdot \frac{M}{N}$ 2-collisions in space $[N]$, which has a time-space tradeoff of $\sim (C_2 \cdot \frac{M}{N}) \cdot \frac{\sqrt{NS}}{S} = C_2 \cdot \frac{M}{\sqrt{N}} \cdot \frac{1}{\sqrt{S}}$. We then keep only those collisions that remain valid in the original output space $[M]$.

However, it is important to note that using memory S does not always yield a $\sqrt{S}$ -fold improvement in time complexity, as in the case of $f_{N,N}$. To achieve a $\sqrt{S}$ -fold improvement, we require that the S 2-collisions found in space $[N]$ correspond to at most C_2 2-collisions in space $[M]$. Therefore, a necessary condition for $\sqrt{S}$ -fold speedup using memory S is

$$C_2 \cdot M \geq S \cdot N.$$

Finding all 2-collisions. The last part concerning 2-collisions is the problem of finding *all* collisions of any function $f_{N,M}$. The time-space tradeoff depends critically on the number of 2-collisions n_2 of $f_{N,M}$. We have already seen that if $n_2 \leq O(N)$, then the number of distinct points reachable by a chain-based method with memory S is approximately $n_d \sim \sqrt{NS}$. This suggests that for any fixed collision pair (x_1, x_2) , the probability that both points are captured by the chains is roughly $\frac{S}{N}$, and there are n_2 such pairs. Therefore, on average, it takes about $\frac{N}{Sn_2}$ executions of the chain-based method to find any pair.

To find all pairs, the total number of executions required is $\sim n_2 \cdot \frac{N}{Sn_2} = \frac{N}{S}$, with each execution costing time $\sqrt{NS}$. This leads to a time-space tradeoff

$$T = \frac{N^{3/2}}{\sqrt{S}}$$

for all functions satisfying $n_2 \leq O(N)$. When $n_2 \gg N$, although the average time to find one 2-collision reduces to $\frac{N}{\sqrt{n_2S}}$, the large number of 2-collisions

means that the total time complexity is at least $\frac{N\sqrt{n_2}}{\sqrt{S}}$. In particular, for random functions, we can upper bound n_2 with overwhelming probability.

As long as the probability of outputting each 2-collision is lower bounded by a constant, repeating for λ times amplifies the success probability to $1 - \text{negl}(\lambda)$.

We emphasize that the time-space tradeoff for finding all 2-collisions is very important. It not only connects to other fundamental problems like *set intersection* and *list distinctness*[4], but also serves as a crucial building block for decision problems of k -collisions, where $k \geq 3$.

2.3 3-collision problems

We are interested in searching for C_3 3-collisions for any $C_3 \geq 1$, and in deciding whether any 3-collision exists for a given function $f_{N,M}$.

Search problem of 3-collisions. The general approach for the 3-collision search problem is to first find a number of 2-collisions, and then apply the multi-target preimage algorithm to search for an additional preimage for some of these 2-collision pairs. This idea was previously proposed in Joux & Lucks's work [27], where a time-space tradeoff for 3-collisions in random functions $f_{N,N}$ was analyzed. By leveraging the randomness of an additional hash function H , we can extend this approach beyond random functions $f_{N,N}$ even $f_{N,M}$ to any functions: the set of 2-collision pairs found using chain-based algorithms behaves like a random subset of all n_2 2-collisions. This allows us to lower bound the success probability of finding 3-collisions in average case. As a result, the time-space tradeoff for finding 3-collisions depends on both the number of 2-collisions n_2 and the number of 3-collisions n_3 , especially the ratio $\frac{n_3}{n_2}$.

For random functions $f_{N,M}$, we can first find S 2-collisions using memory $\sim S$, which takes time $\sim S \cdot \frac{M}{\sqrt{NS}} = \frac{M\sqrt{S}}{\sqrt{N}}$, as discussed in the 2-collision setting. Given these S 2-collisions in the output space $[M]$, the expected time to find C_3 many preimages using the multi-target preimage strategy is $\sim C_3 \cdot \frac{M}{S}$. The best achievable time-space tradeoff under memory S is therefore:

$$T = C_3 \cdot \frac{M}{S},$$

which is the dominating term when $C_3 \cdot \frac{M}{S} \geq \frac{M\sqrt{S}}{\sqrt{N}} \Leftrightarrow S \leq C_3^{2/3} N^{1/3}$. However, we observe that under this condition, we cannot reach the optimal time complexity $C_3^{1/3} M^{2/3}$, as the best we can achieve is $C_3^{1/3} \cdot \frac{M}{N^{1/3}}$. To obtain the optimal time complexity, the number of 2-collisions used must be at least $C_3^{2/3} M^{1/3}$. Furthermore, the optimal time implies that generating these 2-collisions must also be done in time $O\left(C_3^{1/3} M^{2/3}\right)$, which requires memory $S \geq C_3^{2/3} \cdot \frac{M^{4/3}}{N}$.

For non-random functions, a similar analysis applies, except that the time complexity of the multi-target preimage algorithm must be adjusted based on n_2 and n_3 . Suppose we find S random 2-collisions out of the total n_2 . Then

approximately $S \cdot \frac{n_3}{n_2}$ of these are part of some 3-collision. Consequently, the time to find C_3 3-collisions is $T \sim C_3 \cdot \frac{Nn_2}{Sn_3}$, as we randomly search the domain $[N]$ for preimages.

Decision problem of 3-collisions. For any function $f_{N,M}$, to decide whether there exists a 3-collision using space S , we consider leveraging the algorithm for finding all 2-collisions. We restrict our analysis to $n_2 \leq O(N)$, since otherwise the existence of a 3-collision becomes trivial due to the pigeonhole principle.

The decision algorithm proceeds as follows: during the execution of the algorithm for finding all 2-collisions, each time a 2-collision pair is found, it is stored in memory until the memory capacity S is reached. Once full, we scan the entire domain $[N]$ to check whether any of the 2-collisions stored in memory participate in a 3-collision. If none are found, we clear the memory and continue running the 2-collision finding algorithm.

A subtle point is that the approach for finding 2-collision is a randomized Monte Carlo algorithm, so some collisions may be output multiple times. We must therefore show that the total number of outputs remains $O(n_2)$. Obtaining a tight upper-tail bound with exponential decay is difficult—and such a tail may not even exist. To address this, we cap the number of collisions produced in a single execution and show that among $\log \lambda$ runs, at least one outputs all collisions while keeping the total (including repeats) within $O(n_2)$.

The total time complexity consists of two parts: the time for finding all 2-collisions using space S , and the time for checking whether any stored 2-collision completes a 3-collision. The time-space tradeoff for finding all 2-collisions is known from previous 2-collisions section to be $\sim \frac{N^{3/2}}{\sqrt{S}}$ for $n_2 \leq O(N)$.

For the 3-collision checking phase, note that on average, the memory will hold $S \cdot \frac{n_3}{n_2}$ 2-collisions that are part of some 3-collisions. Hence, to expect one such 3-collision, we need to load $\sim \frac{n_2}{Sn_3}$ batches of S 2-collisions into memory during finding all 2-collisions. Each batch requires time N to scan the domain $[N]$ to check if 3-collision exists. Therefore, the checking cost is $\sim \frac{n_2}{Sn_3} \cdot N$.

Putting both parts together, the overall time-space tradeoff becomes:

$$T \sim \frac{N^{3/2}}{\sqrt{S}} + \frac{n_2}{Sn_3} \cdot N.$$

For random functions $f_{N,M}$ with $M = \Theta(N^{3/2})$, we have $n_2 \sim \sqrt{N}$ and $n_3 \sim 1$, so the checking cost is $O\left(\frac{\sqrt{N}}{S} \cdot N\right) = O\left(\frac{N^{3/2}}{S}\right)$. Thus, for any S , the total time is dominated by finding all 2-collisions, and $T = O\left(\frac{N^{3/2}}{\sqrt{S}}\right)$.

2.4 k -collision problems

For the search problems and decision problems of k -collision, we generalize the techniques used for 3-collision. The main difference is that to find C_k k -collisions, we need to find $C_2 \leq S$ 2-collisions given memory S , then find C_3 3-collisions,

C_4 4-collisions, and so on, up to C_{k-1} $(k-1)$ -collisions. By balancing the time complexity for finding C_i i -collisions for each $i \in [2, k]$, we can determine the overall time-space tradeoff. As suggested in the 3-collision case, the optimal time complexity cannot be achieved with the best time-space tradeoff for general $f_{N,M}$, so there will be two tradeoff curves.

For the decision problem, the algorithm for finding 3-collisions can be modified to find k -collisions. Similarly, during the execution of the algorithm for finding all 2-collisions, we store each 2-collision found into memory until approximately $\frac{1}{k}$ of the total memory S is filled. The factor $\frac{1}{k}$ arises because it takes roughly k times more memory to store a k -collision compared to a 2-collision.

Thus, there will be approximately $\frac{S}{k}$ 2-collisions stored in memory. We then scan the input domain $[N]$ to check if any of these $\frac{S}{k}$ 2-collisions are part of a 3-collision, replacing the 2-collisions with the corresponding 3-collisions when found. Afterward, another scan can be performed to obtain 4-collisions, and so on. Overall, it requires approximately $(k-2)N$ time to determine whether any of the $\frac{S}{k}$ 2-collisions are part of some k -collision. If k is small, the time-space tradeoff of deciding k -collisions is $T \sim \frac{N^{3/2}}{\sqrt{S}} + \frac{n_2}{S n_k} \cdot N$.

3 Preliminary and Some Statistics

The functions we considered are functions $f_{N,M}$, $M \geq N$ defined over $[N] = \{0, 1, \dots, N-1\}$, $[M] = \{0, 1, \dots, M-1\}$, we will use $[N]$ to denote this set for the entire paper. For notations, we use $\tilde{O}, \tilde{\Theta}, \tilde{\Omega}$ to ignore the logarithmical factors. Usually, we use S to denote the space complexity where s to denote the number of chains stored in memory S where the length will be denoted as ℓ .

We use n_k , $k \geq 2$ to denote the number of k -collisions for any function $f_{N,M}$. We also use n_d to denote the number of random distinct points generated by the chained algorithm, this will be introduced in this section.

In addition, we show several important statistic that will be use throughout our work, the proofs will be provided with details in [Appendix A](#).

Effective maximum chain length. We define the *effective maximum chain length* as the largest possible chain ℓ length with $\Omega(\ell)$ distinct points that arises with non-negligible probability for single chain. For multi-chain scenario, this refers to constructing s chains with maximum length ℓ and any chain has $\Omega(\ell)$ distinct points. Intuitively, functions with larger n_2 tend to produce shorter effective maximum chain lengths, making ℓ a useful estimator of n_2 . Below, we derive the precise relationship between the effective maximum chain length ℓ and n_2 .

Lemma 2. *Under the single- or multi-chain method using a random hash function $H_{M,N}$, for any function $f_{N,M}$ with $M \geq N$ and n_2 2-collisions, constructing s chains of equal length yields an effective maximum chain length*

$$\ell = O\left(\min\left(\sqrt{\frac{N}{s}}, \frac{N}{\sqrt{n_2 s}}\right)\right).$$

In particular, if $f_{N,M}$ is random so that $n_2 = O(N)$, then $\ell = O(\sqrt{N/s})$.

Remark 4. When $n_2 \gg N$, the bound can be inverted to estimate the number of collisions in terms of the observed effective maximum chain length:

$$\ell = O\left(\frac{N}{\sqrt{n_2 s}}\right) \Rightarrow n_2 = O\left(\frac{N^2}{s \ell^2}\right).$$

Thus, the effective maximum chain length ℓ serves as a practical estimator for n_2 . When $n_2 \ll N$, collisions are rare, and n_2 can instead be estimated by measuring the expected time to find one 2-collision.

Number of distinct points within the chains. Previously, we introduced the *single chain* and *shorter multi-chain* methods. In this work, we aim to compute the probability of finding collisions by deriving a lower bound on the number of *distinct points* across these chains.

The rationale is that collisions within the chains influence the probability of subsequent collisions. However, by considering only the distinct points—which are random because the hash function $H_{M,N}$ is random—we can derive a meaningful lower bound for the collision probability giving the lower bound of number of random distinct points. This approach works because the number of distinct points already accounts for the effects of prior collisions. Thus, as long as the chains contain sufficiently many distinct points with high probability, we can lower bound the probability of finding any specific collision, regardless of the total number of collisions within the chains.

Lemma 3. *Under the chain-based algorithm method, for any function $f_{N,M} : [N] \rightarrow [M]$, $M \geq N$ with n_2 pairs of 2-collision, given a true random hash function $H_{M,N} : [M] \rightarrow [N]$, constructing $s \ll N$ chains $(x_i^j, y_i^j)_{i \in [\ell], j \in [s]}$ with length $\ell = \min\left(\frac{N}{\sqrt{n_2 s}}, \sqrt{\frac{N}{s}}\right)$ by*

$$\begin{aligned} x_1^j &\leftarrow \$[N], y_1^j = f_{M,N}(x_1^j), j \in [s] \\ x_i &= H_{M,N}(y_{i-1}^j), y_i^j = f_{M,N}(x_i^j), i \in [2, \ell], j \in [s] \end{aligned}$$

Denote the number of distinct points x_i^j over the chains as n_d , then

$$\Pr(n_d \geq \frac{1}{4} s \ell) \geq 1 - \exp(-\Omega(s)).$$

Proof. Proof sketch: we lower bound n_d by constructing a “pruned” version (has less distinct points) of the original chain-based construction but decouples the chains, making them independent. See full details at [Appendix A](#). $\square$

Number of 2-collisions for random function $f_{N,M}$. Since we are dealing with random functions for all $M \in [N, N^2]$, we need to compute the number of collisions for random function $f_{N,M}$. We need both the upper bound and lower bound, where lower bound will be used for finding *some* collisions and upper bound will be used for finding *all* collisions.

Lemma 4. Let $f_{N,M} : [N] \rightarrow [M]$ be a random function, and let n_2 denote the number of collisions of $f_{N,M}$. If $M \leq N^2$, then

$$\Pr\left(n_2 \geq \frac{N(N-1)}{4M}\right) \geq 1 - 2\exp(-\Omega(N^2/M)).$$

Moreover, the event $n_2 \leq O(N)$ occurs except for negligible probability for all $M \geq N$. More precisely, for the upper tail, we have $\Pr(n_2 \leq \frac{3N(N-1)}{2M}) \geq 1 - \frac{4}{\sqrt{N}}$ for all $M \ll N^2$. In particular, when $M \sim N^2$, there exists a constant $c > 0$ such that $\Pr(n_2 \leq \frac{cN(N-1)}{2M}) \geq 1 - \frac{4}{N}$.

4 Time-Space Tradeoffs of 2-Collision Problems

Algorithm 1: $\mathcal{A}_{chain}(\ell, s, f_{N,M}, H_{M,N})$, base chain algorithm

```

// Single chain algorithm given low memory
1 if  $s = 1$  then
2   Starting from a random point  $x_1 \leftarrow \$[N]$ , let  $y_1 = f_{N,M}(x_1)$ ;
3   Run the Floyd's cycle finding algorithm to find any collision
    $(x_{co1}, x_{co2})$  within length  $\ell$  and output if exists.
// Multi-chains algorithm given  $S$  memory
4 if  $s > 1$  then
5   Initialize set  $A = \{\}$ ;
6   for  $j = 1$  to  $s$  do
7     Starting from a random point  $x_1^j \in [N]$ , let  $y_1^j = f_{N,M}(x_1^j)$ ;
8     for  $i = 2$  to  $2\ell$  do
9        $x_i^j = H_{M,N}(y_{i-1}^j)$ ,  $y_i^j = f_{N,M}(x_i^j)$ ;
10      if  $i = \ell$  then
11        Store  $(x_{st}^j, y_{end}^j) = (x_1^j, y_\ell^j)$  into  $A$ ;
12      if  $\exists (x_{st}^{j*}, y_{end}^{j*}) \in A$  with  $y_{end}^{j*} = y_i^j$  then
13        Using  $x_{st}^{j*}, x_1^j$  find earliest collision location  $y_{co1}^{j*} = y_{co2}^j$ ;
14        if  $x_{co1}^{j*} \neq x_{co2}^j$  then
15          Output  $(x_{co1}^{j*}, x_{co2}^j)$ ;
16        if  $co2 \leq \ell$  then
17          Replace  $(x_{st}^j, y_{end}^j) = (x_1^j, y_{co2}^j)$  in  $A$ ;
18        Break the inner for loop of  $i$ .
// Take care of the collision within each chain1
19 Run the Floyd's cycle finding algorithm to find any collision
    $(x_{co1}, x_{co2})$  within length  $\ell$ . If exists, output the collision and
   remove the non-distinct points within length  $\ell$ .

```

This case is included primarily for technical reasons: without it, we cannot directly derive a lower bound on the probability of finding each collision pair based on the number of distinct points.

In this section, we explore the time-space tradeoffs for 2-collisions in any function $f_{N,M} : [N] \rightarrow [M]$. We begin with a base chain algorithm—using either a single long chain or multiple shorter chains—that can be used to establish the basic algorithm for searching 2-collisions. We then specialize to random functions, deriving trade-off curves both for finding a fixed number of collisions and for finding all collisions, and we discuss how these results related to known lower bounds. Finally, we extend our analysis in to arbitrary functions.

Using base chain algorithm 1, we derive the time-space tradeoff below.

Lemma 5. *Given oracle access to any function $f_{N,M}$, $M \geq N$ with n_2 2-collisions and a random hash function $H_{M,N}$, there is an algorithm $\mathcal{A}_{base}^2$ runs in time $O\left(\sqrt{\frac{S}{\log M}} \min\left(\sqrt{N}, \frac{N}{\sqrt{n_2}}\right)\right)$ using space S for any $S \geq \Omega(\log M)$ bits, such that for every pair of $x_1 \neq x_2 \in [N]$ with $f_{N,M}(x_1) = f_{N,M}(x_2)$,*

$$\Pr[(x_1, x_2) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N}, S)] \geq \Omega\left(\frac{S}{\log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right)\right),$$

In particular, if we have $a = \frac{S \min(\frac{n_2}{N}, 1)}{2 \log M} \geq 1$, then the probability of outputting $\Theta(a)$ different 2-collisions in one run is $\Omega(1)$. If we have $a < 1$, then the probability of outputting at least one 2-collision in $\frac{1}{a}$ independent runs is $\Omega(1)$.

Proof. The algorithm $\mathcal{A}_{base}^2$ proceeds as follows:

1. If $S \sim \log M$, set $s = 1$ and $\ell = \min\left(\frac{N}{\sqrt{n_2}}, \sqrt{N}\right)$. Otherwise, compute $s = \lceil \frac{S}{2 \log M} \rceil$ and set $\ell = \min\left(\sqrt{\frac{N}{s}}, \frac{N}{\sqrt{n_2 \cdot s}}\right)$.
2. Run $\mathcal{A}_{chain}(\ell, s, f_{N,M}, H_{M,N})$ and output whatever is returned by $\mathcal{A}_{chain}$.

Therefore, the time-space tradeoff of $\mathcal{A}_{base}^2$ matches that of $\mathcal{A}_{chain}$, which is $T = O(s\ell) = O\left(\min\left(\sqrt{N}s, \frac{N\sqrt{s}}{\sqrt{n_2}}\right)\right) = O\left(\sqrt{\frac{S}{\log M}} \min\left(\sqrt{N}, \frac{N}{\sqrt{n_2}}\right)\right)$.

Since the chain elements are outputs of a random function, the set of distinct points visited in one run is uniformly distributed over $[N]$, regardless of whether they originate from a single chain or from multiple chains. Therefore, the probability of detecting a given collision can be bounded in terms of the number of distinct random points. Specifically, for a fixed collision pair (x_1, x_2) , let $E = \{\exists x_{i_1}^{j_1}, x_{i_2}^{j_2} \text{ on some chains such that } x_1 = x_{i_1}^{j_1}, x_2 = x_{i_2}^{j_2}\}$, we have

$$\begin{aligned} \Pr[(x_1, x_2) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N})] &= \Pr[E] \\ &\geq \Pr[E \mid n_d \geq \frac{s\ell}{4}] \Pr(n_d \geq \frac{s\ell}{4}) \sim \left(\frac{s\ell/4}{N}\right)^2 (1 - \exp(-\Omega(s))) \\ &\geq \frac{S}{32 \log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right) = \Omega\left(\frac{S}{\log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right)\right). \end{aligned}$$

To bound the number of collisions, we have to account for dependencies. While $n_{c,2} \geq \Omega(N)$ admits a simpler proof, the general proof requires evaluating these dependencies. Here we provide a sketch here, see [Appendix B](#) for details.

We split into two cases: $n_{c,2} \geq \frac{N^2}{(s\ell)^2}$ (equivalently $a \geq 1$) and $n_{c,2} < \frac{N^2}{(s\ell)^2}$ (equivalently $a < 1$). For both cases, we directly compute the dependency term

$\sum E[X_{i,j}X_{k,\ell}]$ over all collision pairs $(i,j), (k,\ell)$, where $X_{i,j} = \mathbf{1}\{f(i) = f(j)\}$. This sum splits into $\sum E[X_{i,j}]^2$ (pairs sharing no points) and $\sum E[X_{i,j}X_{j,k}]$ (pairs sharing one point). Janson's inequality yields a bound of the form

$$\exp\left(-\frac{\mu^2}{\mu + \mu^2 + \sum E[X_{i,j}X_{j,k}]}\right),$$

where $\mu = E[X]$ and X is the number of collisions.

When $a \geq 1$, the term μ^2 dominates, so it suffices to have $\frac{\sum E[X_{i,j}X_{j,k}]}{\mu^2} = O(1)$, which holds when $n_{c,2} \geq \frac{N^2}{(s\ell)^2}$. When $a < 1$, the term μ dominates, so we require $\frac{\sum E[X_{i,j}X_{j,k}]}{\mu} = O(1)$; this is guaranteed when $n_{c,2} < \frac{N^2}{(s\ell)^2}$, and the small μ is compensated by running the algorithm $\frac{1}{a} = \Omega(\frac{1}{\mu})$ times. $\square$

4.1 2-Collision Problems for Random Functions

Now we study the problem of finding C_2 2-collisions and the problem of finding *all* 2-collisions for random functions $f_{N,M} : [N] \rightarrow [M]$, assuming $M \in [N, N^2]$.

Searching for C_2 collisions. We recall from Lemma 4 that for a random function $f_{N,M}$, the number of 2-collisions satisfies $\Omega(N) \leq n_2 < O(N)$ except with negligible probability. By Lemma 5, the expected time to output one collision equals the running time of $\mathcal{A}_{base}^2$ divided by a , namely

$$T = O\left(\log M \cdot M/(\sqrt{NS})\right),$$

giving a $\sqrt{S}$ -fold speedup using memory S compared to the low-memory case when many collisions are needed.

If the target number of collisions C_2 is smaller than $O\left(\frac{SN}{M \log M}\right)$, then full $\sqrt{S}$ speedup can not be achieved, since one execution of $\mathcal{A}_{base}^2$ already exceeds C_2 . In this case, the optimal tradeoff is achieved with reduced memory $S' < S$, where $\sqrt{S'} \sim \sqrt{\frac{C_2 M \log M}{N}}$.

We summarize our main result for 2-collision search in the following theorem.

Theorem 1. *Given oracle access to any random function $f_{N,M}$, where $M \in \left[\frac{SN}{C_2 \log M}, N^2\right]$ and $O(N)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$, there exists an algorithm $\mathcal{A}_{rd,search}^2$ that outputs at least $C_2 \ll \frac{N^2}{M}$ different 2-collisions of $f_{N,M}$ with probability $1 - \text{negl}(\lambda)$, for any $\lambda = O(\log(N))$. The algorithm achieves time-space tradeoff*

$$T = O\left(\lambda C_2 \sqrt{\log M} \cdot \frac{M}{\sqrt{NS}}\right) = \tilde{O}\left(\frac{C_2 M}{\sqrt{NS}}\right).$$

Proof. The algorithm $\mathcal{A}_{rd,search}^2$ works as follows:

- Repeat the base algorithm $\mathcal{A}_{base}^2$ for $O\left(\lambda \cdot \frac{C_2 M \log M}{NS}\right)$ times. In the u -th execution, provide inputs $(f_{N,M}, H_{M,N}^u, S)$.

To show that the algorithm can output C_2 different collisions, note that the proof of Lemma 5 also applies when considering any subset of collisions of size $n'_{c,2} = \Omega(n_{c,2})$, with $a' = s \cdot \min\left(\frac{n'_2}{N}, 1\right)$. Thus, as long as at least $\Omega(n_{c,2})$ collisions remain uncollected, each run of $\mathcal{A}_{base}^2$ outputs

$$\Omega\left(\frac{S}{\log M} \cdot \frac{n_2}{N}\right) = \Omega\left(\frac{S}{\log M} \cdot \frac{N}{M}\right)$$

new collisions with constant probability, since $n_{c,2} \geq \frac{1}{4} \frac{N^2}{M}$ except negligible probability. Amplifying with λ independent runs gets success probability $1 - \text{negl}(\lambda)$. Hence, before collecting C_2 different collisions, the remaining pool always satisfies $n'_2 \geq n_2 - C_2 = \Omega(n_2)$, so repeating $\mathcal{A}_{base}^2$ for

$$O\left(\lambda \cdot \frac{C_2}{\frac{SN}{M \log M}}\right) = O\left(\lambda \cdot \frac{C_2 M \log M}{NS}\right)$$

runs suffices to obtain C_2 different collisions with probability $1 - \text{negl}(\lambda)$.

The total time complexity is

$$T = O\left(\lambda C_2 \cdot \frac{M \log M}{NS} \cdot \sqrt{\frac{NS}{\log M}}\right) = \tilde{O}\left(\frac{C_2 M}{\sqrt{NS}}\right),$$

provided $M \geq \frac{SN}{C_2 \log M}$, i.e., $C_2 \geq \frac{SN}{M \log M}$. $\square$

Searching for *all* collisions. For the problem of finding *all* 2-collisions, we show that the time-space tradeoff follows a unified curve across all random functions $f_{N,M}$ with $M \geq N$, which is $T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$.

Theorem 2. *Given oracle access to any random function $f_{N,M}$, $M \in [N, N^2]$ and $O(N)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$, there exists an algorithm $\mathcal{A}_{rd,all}^2$ that for every collision (x_1, x_2) of $f_{N,M}$ and any λ*

$$\Pr[(x_1, x_2) \in \mathcal{A}_{rd,all}^2(f_{N,M}, \{H_{M,N}^u\}, S)] = 1 - \text{negl}(\lambda),$$

obtaining the time-space tradeoff $T = O\left(\frac{\lambda N^{3/2} \log^{3/2} M}{\sqrt{S}}\right) = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$.

Proof. The algorithm $\mathcal{A}_{rd,all}^2$ proceeds as follows:

- Repeat the base algorithm $\mathcal{A}_{base}^2$ for $t = O\left(\lambda \cdot \frac{N \log^2 M}{S}\right)$ times. In the u -th run, use input $(f_{N,M}, H_{M,N}^u, S)$ and collect all outputs.

Since we want to lower bound the probability of getting all 2-collisions, we need to take the union bound of the probability that each 2-collision of $f_{N,M}$ will be outputted in at least one of the t executions. So we have

$$\begin{aligned} \Pr[\text{Get all } n_2 \text{ collisions}] &\geq 1 - \sum_{i \in [n_2]} \Pr[\text{not get } i\text{-th collision}] \\ &\geq 1 - \text{negl}(\lambda) \quad \text{if } \Pr[\text{not get } i\text{-th collision}] \leq \frac{\text{negl}(\lambda)}{n_2}, \forall i \in [n_2]. \end{aligned}$$

Furthermore, by Lemma 4, we know that $n_2 \leq O(N)$ except with negligible probability as long as $M \geq N$. Then we have

$$\begin{aligned} &\Pr[\text{not get the } i\text{-th collision in } t \text{ independent runs of } \mathcal{A}_{base}^2] \\ &\leq \left(1 - \Omega\left(\frac{S}{N \log M}\right)\right)^t \\ &= \frac{\text{negl}(\lambda)}{n_2} \Leftrightarrow \frac{St}{N \log M} = O(\lambda \log n_2) \Leftrightarrow t = O\left(\lambda \frac{N \log^2 M}{S}\right). \end{aligned}$$

Thus, choosing $t = O\left(\lambda \cdot \frac{N \log^2 M}{S}\right)$ completes the proof. $\square$

Conditional Optimality. Our results above show that, for random $f_{N,M}$, $M \geq N$, achieving the optimal time complexity $T = \tilde{O}(\sqrt{M})$ for finding *one* 2-collision requires space $S = \tilde{\Omega}(M/N)$. This indicates that finding collisions in $f_{N,M}$ may be strictly harder than in $f_{M,M}$ in terms of time-space tradeoffs where $f_{M,M}$ only requires $S^* \sim O(\log M)$ to achieve $T = \tilde{O}(\sqrt{M})$.

Dinur [17] proved an unconditional lower bound on time-space tradeoffs for collisions in random $f_{N,N}$. Building on this, we show that our algorithm for $f_{N,M}$ is conditionally optimal. We fix an efficient invertible mapping (C_f, D_f) between $[M]$ and a subset $S \subseteq [N] \times \lceil [M/N] \rceil$, with $D_f : [M] \rightarrow S$, $C_f : S \rightarrow [M]$, and $D_f(z) = (D_f^0(z) = x, y)$. Hence

$$\forall z \in [M], D_f(z) = (x, y) \text{ if and only if } C_f(x, y) = z, x \in [N], y \in \lceil [M/N] \rceil.$$

For any algorithm $\mathcal{A}$ that outputs C_2 different collisions of $f_{N,M} : [N] \rightarrow [M]$, if there exist an efficient (C_f, D_f) that $\mathcal{A}$ can also output

$$\Omega\left(C_2 \cdot \frac{M}{N}\right)$$

collisions of $D_f^0 \circ f_{N,M} : [N] \rightarrow [N]$, then the time-space tradeoff must satisfy

$$T^2 S \geq \tilde{\Omega}\left(C_2^2 \cdot \frac{M^2}{N}\right).$$

The reduction is as follows. If $N \mid M$, C_f reduces finding C_2 collisions in random $f_{N,N}$ to $\mathcal{A}$ via $f_{N,M}$ constructed from $f_{N,N}$ and an independent random map $[N] \rightarrow [M/N]$. If $N \nmid M$, we reduce to the divisible case with polylogarithmic overhead by defining $f_{N,M}$ from independent random oracles

$$f_{N,N} : [N] \rightarrow [N], \quad f'_{N, \lceil [M/N] \rceil} : [N] \rightarrow \lceil [M/N] \rceil, \quad H_{N,M} : [N] \rightarrow [M]$$

via

$$f_{N,M}(x) = \begin{cases} C_f(f_{N,N}(x), f'_{N, \lceil M/N \rceil}(x)), & < M, \\ H_{N,M}(x), & \text{otherwise.} \end{cases}$$

Since $f_{N,M}$ is random with $M \geq N$, the largest collision component has size $\Theta(\frac{\log N}{\log \log N})$. Thus, among any C_2 collisions, at least $\Omega(C_2/(\log N)^2)$ hit distinct images. Each also yields an $f_{N,N}$ -collision with probability at least $p^2 \geq (M/(M+N))^2 \geq 1/4$, so by a Chernoff bound a constant fraction survive with high probability.

Remark 5. Our optimality is *conditional*: it excludes asymptotic improvements for algorithms based on generating projected/partial collisions and lifting them, but does not prove general optimality. In particular, Dinur's method for $f_{N,N}$ does not naturally extend to $f_{N,M}$ when $M = N^{1+c}$, $c > 0$. Any lower bound stronger than $T^2 S \geq \tilde{\Omega}(C_2^2 M)$ would be especially interesting, as it would separate the $[N] \rightarrow [M]$ and $[M] \rightarrow [M]$ cases.

4.2 2-Collision Problems for Arbitrary Functions

The results established for random functions extend naturally to arbitrary functions, where the complexity depends on the number of 2-collisions n_2 .

Theorem 3. *Given oracle access to any function $f_{N,M}$, $M \geq N$ with n_2 2-collisions and $O(N)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$, there exists an algorithm $\mathcal{A}_{\text{arb,search}}^2$ outputs at least $C_2 \ll n_2$ pairs 2-collisions of $f_{N,M}$ with probability $1 - \text{negl}(\lambda)$ for any λ , obtaining the following time-space tradeoff:*

$$T = \begin{cases} O\left(C_2 \lambda \sqrt{\log M} \cdot \frac{N^{3/2}}{n_2 \sqrt{S}}\right) & \text{if } n_2 \leq N, \\ O\left(C_2 \lambda \sqrt{\log M} \cdot \frac{N}{\sqrt{n_2 S}}\right) & \text{if } n_2 > N, \end{cases}$$

assuming the available space satisfies $S \geq C_2 \log M$.

Proof. Let $\mathcal{A}_{\text{arb,search}}^2$ run $\mathcal{A}_{\text{base}}^2$ for $t = O\left(\frac{\lambda C_2}{\frac{S}{\log M} \cdot \min(\frac{n_2}{N}, 1)}\right)$ times using independent hash functions $H_{M,N}^u, u \in [t]$. It then follows from Theorem 1. $\square$

We now turn to the problem of finding *all* 2-collisions for arbitrary functions. We show that for all functions $f_{N,M}$ with $n_2 \leq O(N)$, the time-space tradeoff remains bounded by the same curve as in the random case, namely $\tilde{O}(N^{3/2}/\sqrt{S})$.

Theorem 4. *Given oracle access to any function $f_{N,M}$, $M \geq N$ with n_2 2-collisions and $O(N^2)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N^2)]}$, there exists an algorithm $\mathcal{A}_{\text{arb,all}}^2$ outputs all 2-collisions of $f_{N,M}$ with probability $1 - \text{negl}(\lambda)$ for any λ , obtaining the following time-space tradeoff:*

$$T = \begin{cases} O\left(\lambda \log^{3/2} M \cdot \frac{N^{3/2}}{\sqrt{S}}\right) & \text{if } n_2 \leq N, \\ O\left(\lambda \log^{3/2} M \cdot \frac{N \sqrt{n_2}}{\sqrt{S}}\right) & \text{if } n_2 > N. \end{cases}$$

Proof. The algorithm $\mathcal{A}_{arb,all}^2$ performs t independent executions of $\mathcal{A}_{base}^2$ using $(f_{N,M}, H_{M,N}^u, S)$ as input. The goal is to choose t such that all n_2 collision pairs are found with probability $1 - \text{negl}(\lambda)$. When $n_2 \leq O(N)$, the time-space tradeoff for finding all 2-collisions can be derived similarly to the random function case by choosing t as $O\left(\lambda \cdot \frac{N \log M \cdot \log n_2}{S}\right) = O\left(\lambda \cdot \frac{N \log^2 M}{S}\right)$.

When $n_2 \gg N$, for each pair, the probability of not finding the pair is $\left(1 - \Omega\left(\frac{S}{n_2 \log M}\right)\right)^t$. Similar to Theorem 2, to take the union bound of all n_2 collisions, we have to choose t such that

$$\left(1 - \Omega\left(\frac{S}{n_2 \log M}\right)\right)^t = \frac{\text{negl}(\lambda)}{n_2}, \quad n_2 \leq N^2,$$

which is satisfied when $t = \Theta\left(\lambda \cdot \frac{n_2 \log M \log n_2}{S}\right)$. Then the tradeoff is

$$T = t \cdot O\left(\sqrt{\frac{S}{\log M}} \cdot \frac{N}{\sqrt{n_2}}\right) = O\left(\lambda \log^{3/2} M \cdot \frac{N \sqrt{n_2}}{\sqrt{S}}\right).$$

□

5 Time-Space Tradeoffs of 3-Collision Problems

In this section, we establish time-space tradeoffs for 3-collision problems in a function $f_{N,M} : [N] \rightarrow [M]$. We consider both the search problems and decision problems for random functions and arbitrary functions with n_2 2-collisions and n_3 3-collisions. In particular, for decision problem of random functions $f_{N,M}$, We focus on the non-trivial case $M = \tilde{\Theta}(N^{3/2})$, where the probability of the existence of a 3-collision is not negligible and not overwhelming.

5.1 Random Functions

We firstly show the results for search problem.

Theorem 5. *Given oracle access to any random function $f_{N,M}$, $M \in [N, N^{3/2}]$ and $O(N)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$, there is an algorithm $\mathcal{A}_{rd,search}^3$ that can output C_3 different 3-collisions with probability $1 - \text{negl}(\lambda)$, achieving time-space tradeoff $T = \tilde{O}\left(\frac{C_3 M}{S}\right)$ for any $S \leq C_3^{2/3} N^{1/3}$.*

Proof. The algorithm 2 is shown above. Given space S , we use $\frac{S}{2}$ to store $\sim \frac{S}{6 \log M}$ 2-collisions where the rest of space is used to store the found 3-collisions. From earlier analysis, we know that creating $C'_2 = \Omega\left(\frac{SN}{M \log M}\right)$ 2-collisions requires λ executions of $\mathcal{A}_{base}^2$ with success probability at least $1 - \text{negl}(\lambda)$. Consequently, creating a total of $\frac{S}{6 \log M}$ different 2-collisions (to fill storage) requires $O\left(\frac{S}{\frac{S}{C'_2}} \cdot \lambda\right) = O\left(\frac{S}{6 \log M} \cdot \frac{M \log M}{SN} \cdot \lambda\right) = O\left(\frac{M}{N} \cdot \lambda\right)$ executions of $\mathcal{A}_{base}^2$. Each

Algorithm 2: $\mathcal{A}_{rd,search}^3(f_{N,M}, \{H_{M,N}\}, S)$ for searching 3-collisions

```

1 Initialize the set of 2-collision as  $A = \{\}$ , let  $n_{2,col} = 0, u = 1$ ;
2 while  $n_{2,col} < \lfloor \frac{S}{6 \log M} \rfloor$  do
3   for every  $(x_1, x_2; y) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N}^u, \frac{S}{2})$  do
4     if there is no  $(x'_1, x'_2, y) \in A$  then
5        $\lfloor$  Store  $(x_1, x_2, y)$  into  $A$ ,  $n_{2,col} = n_{2,col} + 1$ ;
6      $\rfloor$   $u = u + 1$ ;
7 Initialize the set of 3-collisions as  $A_3 = \{\}$ , let  $n_{3,col} = 0$ ;
8 for  $x^j \in [N]/A$  do
9   Compute  $y^j = f_{N,M}(x^j)$ ;
10  if there is some  $(x_1, x_2; y^j) \in A$  and  $x_1 \neq x_2 \neq x^j$  then
11    Put  $(x_1, x_2, x^j; y^j)$  into  $A_3$ , let  $n_{3,col} = n_{3,col} + 1$ ;
12    if  $n_{3,col} = C_3$  then
13       $\lfloor$  Output  $A_3$  and terminate the algorithm.

```

such execution costs $O\left(\sqrt{\frac{NS}{\log M}}\right)$ time, so the total time complexity for generating 2-collisions is

$$T'' = O\left(\frac{M}{N} \cdot \lambda \cdot \sqrt{\frac{NS}{\log M}}\right) = \tilde{O}\left(\frac{M}{N} \cdot \sqrt{NS}\right).$$

Given $\frac{S}{6 \log M}$ different 2-collision pairs, we analyze the cost to find C_3 3-collisions. Assume we sample values $x^j \in [N] \setminus A$ and compute $y^j = f_{N,M}(x^j)$. The probability of finding at least one 3-collision among t such samples is:

$$\begin{aligned} & \Pr[E : \{\exists j \in [t], \exists (x_1, x_2, y^j) \in A, x^j \neq x_1 \neq x_2, f_{N,M}(x^j) = y^j\}] \\ &= 1 - \left(1 - \frac{\frac{S}{6 \log M}}{M}\right)^t = 1 - \text{negl}(\lambda) \Leftrightarrow t = \Theta\left(\frac{\lambda M \log M}{S}\right). \end{aligned}$$

Thus, to find C_3 different 3-collisions, the total number of samples required is

$$T' = O(C_3 \cdot t) = O\left(\frac{C_3 \lambda M \log M}{S}\right) = \tilde{O}\left(\frac{C_3 M}{S}\right).$$

The total time complexity is therefore the sum of the time to generate 2-collisions and the time to upgrade them to 3-collisions: $T = T' + T'' = \tilde{O}\left(\frac{C_3 M}{S} + \frac{M}{N} \cdot \sqrt{NS}\right)$. By setting $T'' \leq T'$, which gives:

$$\frac{M}{N} \cdot \sqrt{NS} \leq \frac{C_3 M}{S} \Rightarrow S \leq \tilde{\Theta}\left(C_3^{2/3} N^{1/3}\right).$$

which completes the proof. $\square$

Conditional Minimal Memory for Optimal Time Complexity. We observe that for $N \ll M \leq N^{3/2}$, the optimal time complexity for finding a single or C_3 many 3-collisions cannot be achieved under the previously derived time-space tradeoffs. The known optimal time complexity is $T_{opt} = \tilde{O}(C_3^{1/3} M^{2/3})$. However, under tradeoff above, the best achievable time using time-space tradeoff is

$$T = \tilde{O}\left(C_3^{1/3} \cdot \frac{M}{N^{1/3}}\right), \quad \text{when } S = \tilde{\Theta}(C_3^{2/3} N^{1/3}).$$

This raises the natural question: what is the *minimal* memory required to achieve the optimal time T_{opt} ?

By leveraging the multi-target preimage algorithm, we require approximately

$$C_2^* = \frac{C_3 M}{T_{opt}} = \frac{C_3 M}{C_3^{1/3} M^{2/3}} = C_3^{2/3} M^{1/3}$$

different 2-collisions of $f_{N,M}$ to find C_3 3-collisions within optimal time. To generate this many 2-collisions in time T_{opt} , we must satisfy

$$C_3^{1/3} M^{2/3} = \tilde{O}\left(C_2^* \frac{M}{\sqrt{NS}}\right), \quad C_2^* = C_3^{2/3} M^{1/3} \iff S = \tilde{\Omega}\left(C_3^{2/3} \cdot \frac{M^{4/3}}{N}\right).$$

Similar to our previous analysis, by following Dinur's analysis [17], this bound is conditionally optimal: any algorithm capable of producing $\sim M^{4/3}/N$ projected/partial 2-collisions (explicitly or implicitly, via its structure) from some component $f'_{N,N}$ must use at least this much memory to achieve T_{opt} .

Time-Space Tradeoffs Beyond $C_3^{2/3} N^{1/3}$. The above analysis can be extended to the any available memory lies in $S \in [C_3^{2/3} N^{1/3}, C_3^{2/3} \cdot \frac{M^{4/3}}{N}]$. In this range, let C_2^* denote the number of 2-collisions to be generated, we should not let $C_2^* \sim S$. We balance the time to generate them and the time to upgrade to 3-collisions by equating $T'' = \tilde{O}(\frac{M}{\sqrt{NS}} \cdot C_2^*) \sim T' = \tilde{O}(\frac{C_3 M}{C_2^*})$. Solving for the balanced time, we obtain the new time-space tradeoff:

$$T = \tilde{O}\left(\frac{C_3^{1/2} M}{(NS)^{1/4}}\right), \quad \text{for } S \in [C_3^{2/3} N^{1/3}, C_3^{2/3} \cdot \frac{M^{4/3}}{N}].$$

Decision Problem of Random 3-Collision. During our technical overview, we have introduced our high-level idea where we first construct a set of 2-collisions, then we scan the whole space to find if these 2-collisions have any other preimage that can lead to a 3-collision. If we cannot find such a 3-collision in this set, we clear the set and construct a set of new 2-collisions. Now we state our result for the non-trivial case when $M = \tilde{\Theta}(N^{3/2})$.

Theorem 6. *Given oracle access to any random function $f_{N,M}$, $M = \tilde{\Theta}(N^{3/2})$, and $O(N)$ random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$. Let $D_3 = \mathbf{yes}$ if a 3-collision exists for $f_{N,M}$, and $D_3 = \mathbf{no}$ otherwise. There is an algorithm $\mathcal{A}_{rd,decision}^3$ such that*

$$\Pr[D_3 = \mathcal{A}_{decision}^3(f_{N,M}, \{H_{M,N}^u\}, S)] \geq 1 - \text{negl}(\lambda)$$

for any λ , with time-space tradeoffs

$$T = O\left(\lambda \log^{3/2} M \cdot \frac{N^{3/2}}{\sqrt{S}}\right) = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right).$$

Proof. The algorithm 3 is shown below.

We note that every 3-collision inherently contains some 2-collisions. By Theorem 2, every 2-collision component of a 3-collision can be found using the algorithm $\mathcal{A}_{rd,all}^2$ with probability $1 - \text{negl}(\lambda)$. In the decision algorithm $\mathcal{A}_{rd,decision}^3$, for each 2-collision found, we perform a full-space scan to check whether it extends to a 3-collision. Thus, if a 3-collision exists, $\mathcal{A}_{rd,decision}^3$ outputs **yes** with probability $1 - \text{negl}(\lambda)$; if no 3-collision exists, the output is definitely **no**.

Algorithm 3: Algorithm $\mathcal{A}_{rd,decision}^3$ for decide if 3-collision exists

```

1 Initialize the set of 2-collision as  $A = \{\}$ , let  $n_{2,col} = 0$ ;
2 Execute  $\mathcal{A}_{rd,all}^2(f_{N,M}, \{H_{M,N}^u\}, \frac{S}{2})$ ;
3 for every  $(x_1, x_2; y) \in \mathcal{A}_{rd,all}^2(f_{N,M}, H_{M,N}^u, \frac{S}{2})$  do
4   if  $n_{2,col} = \lfloor \frac{S}{6 \log M} \rfloor$  then
5     for  $x^j \in [N]/A$  do
6       Compute  $y^j = f_{N,M}(x^j)$ ;
7       if there is some  $(x'_1, x'_2; y^j) \in A$  and  $x'_1 \neq x'_2 \neq x^j$  then
8         Output yes and stop the algorithm.;
9     Reset the  $n_{2,col} = 0$ ;
10  if  $n_{2,col} < \lfloor \frac{S}{6 \log M} \rfloor$  then
11    if there is some  $(x'_1, x'_2; y) \in A$  and  $(x_1, x_2) \neq (x'_1, x'_2)$  then
12      Output yes and stop the algorithm;
13    if there is no  $(x'_1, x'_2; y) \in A$  then
14      Store  $(x_1, x_2; y)$  into  $A$ ,  $n_{2,col} = n_{2,col} + 1$ ;
15 Output no.
```

The extra cost beyond $\mathcal{A}_{rd,all}^2$ comes from testing whether each reported 2-collision extends to a 3-collision. Let X_{all} be the total number of 2-collision *outputs* (counting repeats) produced by $\mathcal{A}_{rd,all}^2$. Unlike the lower-tail analysis, an upper tail with negligible failure is harder to obtain directly; instead we cap the number of 2-collision outputs per run without affecting correctness.

By Theorem 2, the procedure uses $t = O(\lambda \cdot \frac{N \log^2 M}{S})$ runs of $\mathcal{A}_{base}^2$. By Lemma 5, in any run the algorithm outputs $O(\frac{S n_2}{N \log M})$ 2-collisions with constant probability. We know that $E[X_{all}] = \lambda n_2 \log M$, therefore

$$\Pr\left(X_{all} \leq O(\lambda n_2 \log M)\right) \text{ is a constant}$$

It means that λ runs of $\mathcal{A}_{rd,all}^2$ must have one of it output $O(\lambda n_2 \log M)$ 2-collisions. Moreover, each run of $\mathcal{A}_{rd,all}^2$ outputs *all* available 2-collisions except with negligible probability, so imposing a per-run cap on the number of 2-collision

outputs does not affect correctness. We therefore execute $\mathcal{A}_{rd,decision}^3$ in a capped mode, checking up to $\lambda n_c \log M$ collisions per run, and repeat it for at most λ runs until one completes within the capped number of 2-collision outputs.

Thus, the total time complexity of $\mathcal{A}_{decision}^3$ is:

$$T = O\left(\lambda^2 \cdot \frac{N^{3/2} \log^{3/2} M}{\sqrt{S}}\right) + O\left(\frac{\lambda^2 \cdot \frac{N^2 \log M}{S}}{\frac{6 \log M}{\sqrt{S}}} \cdot N\right) = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right),$$

given $M = \tilde{\Theta}(N^{3/2})$. □

5.2 Arbitrary Functions

For any function $f_{N,M}$ with number of 2-collisions n_2 and number of 3-collisions n_3 , we can derive time-space tradeoffs for search and decision problems. In particular, for the decision problem, we only consider the case when $n_2 = O(N)$, as otherwise the decision problem will be trivial.

Search for 3-Collisions in Arbitrary Functions. Given space S , to find C_3 many 3-collisions, we first generate $\lfloor S/(6 \log M) \rfloor$ 2-collisions. By Theorem 3, the time required to obtain these C_2 2-collisions is:

$$T'' = \begin{cases} \tilde{O}\left(C_2 \cdot \frac{N^{3/2}}{n_2 \sqrt{S}}\right) = \tilde{O}\left(\sqrt{S} \cdot \frac{N^{3/2}}{n_2}\right) & \text{if } n_2 \leq N, \\ \tilde{O}\left(C_2 \cdot \frac{N}{\sqrt{n_2 S}}\right) = \tilde{O}\left(\sqrt{S} \cdot \frac{N}{\sqrt{n_2}}\right) & \text{if } n_2 > N. \end{cases}$$

Since these 2-collisions are generated using a random hash function $H_{M,N}$, each one extends to a 3-collision with probability approximately n_3/n_2 . The number of samples t needed to find a 3-collision with probability $1 - \text{negl}(\lambda)$ is:

$$1 - \left(1 - \frac{n_3}{n_2} \cdot \frac{1}{N} \cdot \left\lfloor \frac{S}{6 \log M} \right\rfloor\right)^t = 1 - \text{negl}(\lambda) \Rightarrow t = \tilde{O}\left(\frac{N n_2}{S n_3}\right).$$

Therefore, the total time-space tradeoff for finding C_3 3-collisions becomes:

$$T = \tilde{O}\left(\frac{C_3 N}{S} \cdot \frac{n_2}{n_3}\right), \quad \text{where } S \leq \begin{cases} \tilde{\Theta}\left(\left(\frac{C_3 n_2^2}{n_3}\right)^{2/3} \cdot N^{-1/3}\right) & \text{if } n_2 \leq N, \\ \tilde{\Theta}\left(\frac{C_3^{2/3} n_2}{n_3^{2/3}}\right) & \text{if } n_2 > N. \end{cases}$$

Decision Problem for 3-Collisions. Recall from prior results that finding all 2-collisions for arbitrary $f_{N,M}$ when $n_2 \leq O(N)$ has time-space tradeoff $\tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}}\right)$. To decide whether any 3-collision exists, we adopt the same approach as in the random function setting: we store $\sim S/\log M$ 2-collisions found during the process and scan the domain to check if any extend to 3-collisions.

Following the same analysis as in Theorem 6, we know we can put a cap on the number of 2-collisions we found so that we only check at most $\tilde{O}(n_2)$ 2-collisions

by running λ times of the algorithm. Given n_3 3-collisions, the probability that a stored 2-collision is part of one 3-collision is

$$1 - \left(1 - \frac{S/\log M}{n_2}\right)^{n_3} \approx \frac{n_3 \cdot S}{n_2 \log M}.$$

Then the expected number of full-domain checks required to find a 3-collision is $\tilde{O}\left(\frac{n_2 \log M}{S n_3}\right)$, each taking $O(N)$ time. The total time complexity becomes:

$$T = \tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}} + \frac{N n_2}{S n_3}\right) \in \left[\frac{N^{3/2}}{\sqrt{S}}, \frac{N^2}{S}\right].$$

Hardness of 3-Element Distinctness. Our results shows the hardness of 3-element distinctness may depend the number of 2-collisions n_2 and 3-collisions n_3 . Notably, we show that the decision problems for 2-collision and 3-collision over random functions $f_{N,M}$ exhibit the same time-space tradeoff (up to logarithmic factors), provided a negligible failure probability is allowed.

The *hardest* case for 3-element distinctness seems to be the case when $n_2 = O(N)$ and $n_3 = O(1)$. Although it is relatively easy to find a 2-collision, verifying whether it extends to a 3-collision requires scanning the entire domain. For this case with very low memory $S \sim \log M$, no known algorithm achieves better than $O(N^2)$ time or a time-space tradeoff better than $O(N^2/S)$.

Estimating the Worst Case for 3-Element Distinctness. The time-space tradeoff for 3-element distinctness in any function $f_{N,M}$ is closely tied to n_2 and n_3 . While estimating n_3 is generally difficult, it is feasible to estimate n_2 through single chain length. This gives a efficient way to estimate the *worst-case* time-space tradeoff for the 3-element distinctness problem on arbitrary functions.

6 Time-Space Tradeoffs for k -collision Problems

Now we extend our results to any k -collision for relatively small k . Previously, Joux & Lucks [27] also extended their results to any k -collision for the random function $f_{N,N} : [N] \rightarrow [N]$. However, they didn't provide the time-space tradeoffs even for $f_{N,N}$, where they only show that for any k -collision, to get the optimal time complexity $\tilde{O}(N^{\frac{k-1}{k}})$, there is an algorithm using space $\tilde{O}(N^{\frac{k-2}{k}})$.

6.1 Random Functions

Decision Problems. We begin with the decision version, as it closely resembles the decision problem for 3-collisions. As before, we focus on the non-trivial case where the output space size satisfies $M = \widetilde{\Theta}(N^{\frac{k}{k-1}})$.

Theorem 7. *Given oracle access to any random function $f_{N,M}$, $M = \widetilde{\Theta}(N^{\frac{k}{k-1}})$, and $O(N)$ random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$. Let $D_k = \mathbf{yes}$ if k -collision exists for $f_{N,M}$ and $D_k = \mathbf{no}$ otherwise. There is an algorithm $\mathcal{A}_{rd,decision}^k$ that*

$$\Pr[D_k = \mathcal{A}_{rd,decision}^k(f_{N,M}, \{H_{M,N}^u\}, S)] \geq 1 - \text{negl}(\lambda)$$

for any λ , with time-space tradeoffs $T = \tilde{O}(\frac{N^{3/2}}{\sqrt{S}} + \frac{N^{\frac{2k-3}{k-1}}}{S})$.

Proof. The algorithm $\mathcal{A}_{rd,decision}^k$ proceeds similarly to $\mathcal{A}_{rd,decision}^3$, as follows:

1. Execute $\mathcal{A}_{all}^2$ with input $(f_{N,M}, \{H_{M,N}^u\}, S/2)$, and store any 2-collisions found into a set A until the memory is filled with $\lfloor \frac{S}{2k \log M} \rfloor$ 2-collisions.
2. Scan the entire domain $[N]$ to identify any 3-collisions containing 2-collisions in A . Replace the 2-collisions in A with any 3-collisions found. Repeat this process iteratively: at each step, scan $[N]$ to upgrade $(i-1)$ -collisions in A to i -collisions, for $i = 4, \dots, k$, always discarding previous lower-order collisions.

We have already bounded the time complexity of Step 1 as $T' = \tilde{O}(\frac{N^{3/2}}{\sqrt{S}})$.

We can take the strategy from Theorem 6 to bound the number of 2-collisions so we need to check at most $O(\lambda^2 \cdot \frac{N^2 \log M}{M})$. We now bound the time complexity of Step 2. Each upgrade step scans the full space $[N]$ and checks each stored collision, of which there are at most $\frac{S}{2k \log M}$. Therefore, given $M = \widetilde{\Theta}(N^{k/(k-1)})$, the time complexity for upgrading step is at most

$$T'' = O((k-2) \cdot N \cdot \frac{\lambda^2 N^2 \log M/M}{S/(2k \log M)}) = \tilde{O}(\frac{N^3}{MS}) = \tilde{O}(\frac{N^{2-\frac{1}{k-1}}}{S}).$$

Unlike the 3-collision case, the first-step cost is not always dominant,

$$T' = T'' \iff \frac{N^{3/2}}{\sqrt{S}} = \frac{N^{2-\frac{1}{k-1}}}{S} \iff S = \tilde{\Omega}(N^{1-\frac{2}{k-1}}).$$

Therefore, the total time-space tradeoff is:

$$T = \begin{cases} \tilde{O}(\frac{N^{3/2}}{\sqrt{S}}) & \text{if } S \geq \tilde{\Omega}(N^{\frac{k-3}{k-1}}), \\ \tilde{O}(\frac{N^{\frac{2k-3}{k-1}}}{S}) & \text{if } S \leq \tilde{O}(N^{\frac{k-3}{k-1}}). \end{cases}$$

(-) *Two-stage approach.* **Thanks to one of the reviewers** who pointed out that for the decision problem, we can first store many 2-collisions in memory. Instead of iteratively upgrading 2-collisions into k -collisions through intermediate $3, 4, \dots, k-1$ collisions, we can directly scan the whole space to count the number of preimages of these 2-collisions, i.e., their collision profiles. Even for the problem we must also output the k -collision, the algorithm can be arranged in two stages: after detecting points with at least k preimages, we perform one additional scan of the space to recover the corresponding k -collision in extra time $O(N)$. This method improves the bound for T'' as follows.

- We can now store up to $\frac{S}{2 \log M}$ 2-collisions in memory, rather than only $\frac{S}{2k \log M}$, so the number of required scans decreases by a factor of k .
- Each storage requires only one scan instead of $k-2$ scans.

Therefore, the two-stage method achieves

$$T''_{\text{two-stage}} = O\left(N \cdot \frac{\lambda N^2 \log M/M}{S/(2 \log M)} + N\right) = O\left(2\lambda \log^2 M \cdot \frac{N^3}{MS}\right),$$

compared to the previous $T'' = O\left(2k^2 \lambda \log^2 M \cdot \frac{N^3}{MS}\right)$. $\square$

Other Non-Trivial Decision Problems of k -Collision for $f_{N,M}$. For the decision problem of k -collision, when k is not a small constant but $k \sim \frac{\log N}{\log \log M}$, we can also no longer determine the existence of a k -collision with overwhelming probability without any computation. The algorithm described above can still solve this decision problem with the following time-space tradeoff

$$\tilde{O}\left(\frac{N^{3/2}}{\sqrt{S}} + \frac{N^3}{MS}\right).$$

When $M \geq \Omega(N^{3/2})$, the first term dominates, resulting in tradeoff $\tilde{O}(N^{3/2}/\sqrt{S})$. When $M < O(N^{3/2})$, the dominating term depends on the memory size S .

Searching for C_k k -collisions. To find one or C_k k -collisions, the basic idea is to create C_2 2-collisions, and use C_2 2-collisions for multi-target preimage algorithm to find C_3 3-collision, $\dots$, until we get C_k k -collisions. As a result, the time-space tradeoffs should balance the time complexity of finding C_t t -collisions for $t \in [2, k]$.

Theorem 8. *Given oracle access to any random function $f_{N,M}$, $M \in [N, N^{k/(k-1)}]$, and $O(N)$ independent random hash functions $\{H_{M,N}^u\}_{u \in [O(N)]}$, there is an algorithm $\mathcal{A}_{rd,search}^k$ that can output C_k different k -collisions with probability $1 - \text{negl}(\lambda)$, achieving time-space tradeoff $T = \tilde{O}\left(C_k^{1/(k-2)} \cdot \frac{M}{S^{1/(k-2)}}\right)$ for any $S \leq \tilde{O}\left(C_k^{2/k} \cdot N^{(k-2)/k}\right)$.*

Proof. The algorithm $\mathcal{A}_{rd,search}^k$ is shown below

Given space S , we first generate approximately $\frac{S}{2k \log M}$ different 2-collisions which requires time $T_2 = O\left(\frac{M}{N} \cdot \lambda \cdot \sqrt{\frac{NS}{\log M}}\right)$.

Using the multi-target preimage technique, we know that given C_t many t -collisions, the time to find C_{t+1} many $(t+1)$ -collisions with overwhelming probability is $T_{t+1} = O\left(\frac{\lambda C_{t+1} M}{C_t}\right)$.

To minimize the total time complexity, we balance the cost across all stages:

$$\frac{M}{N} \cdot \sqrt{NS} \sim \frac{C_3 M}{S} \sim \frac{C_4 M}{C_3} \sim \dots \sim \frac{C_k M}{C_{k-1}}.$$

This gives $S \leq \tilde{O}\left(C_k^{2/k} \cdot N^{(k-2)/k}\right)$, $C_t = \tilde{O}\left(C_k^{(t-2)/(k-2)} \cdot S^{(k-t)/(k-2)}\right)$.

Algorithm 4: Algorithm $\mathcal{A}_{C_k}^k$ for finding C_k k -collisions

```

1 Initialize the set of 2-collision as  $A = \{\}$ , let  $n_{2,col} = 0, u = 1$ ;
2 while  $n_{2,col} < \lfloor \frac{S}{2k \log M} \rfloor$  do
3   Run  $\mathcal{A}_{rd,search}^2(f_{N,M}, H_{M,N}^u, \frac{S}{2})$ ;
4   for every  $(x_1, x_2; y) \in \mathcal{A}_{rd,search}^2(f_{N,M}, H_{M,N}^u, \frac{S}{2})$  do
5     if there is some  $(x'_1, x'_2; y) \in A$  and  $(x_1, x_2) \neq (x'_1, x'_2)$  then
6       Store  $(Unique(x_1, x_2, x'_1, x'_2); y)$  into  $A$ ,  $n_{2,col} = n_{2,col} + 1$ ;
7     if there is no  $(x'_1, x'_2; y) \in A$  then
8       Store  $(x_1, x_2; y)$  into  $A$ ,  $n_{2,col} = n_{2,col} + 1$ ;
9    $u = u + 1$ 
10 for  $t = 2$  to  $k - 1$  do
11    $C_{find} = 0$ ;
12   for  $x^j \in [N]/A$  do
13     Compute  $y^j = f_{N,M}(x^j)$ ;
14     if  $\exists (x_1, x_2, \dots, x_t; y^j) \in A$  and  $x_i \neq x^j, i \in [1, t]$  then
15       Replace  $(x_1, x_2, \dots, x_t; y^j)$  by  $(x_1, x_2, \dots, x_t, x^j; y^j)$ ;
16        $C_{find} = C_{find} + 1$ ;
17   if  $t = k - 1$  and  $C_{find} = C_k$  then
18     Output  $C_{find}$  and stop the algorithm.

```

Then the time-space tradeoff is $T = \tilde{O}\left(C_k^{1/(k-2)} \cdot \frac{M}{S^{1/(k-2)}}\right)$, for any $S \leq \tilde{O}\left(C_k^{2/k} N^{(k-2)/k}\right)$.

□

Minimal Memory for Optimal Time Complexity. As in the case of 3-collision, the time-space tradeoff derived for general k -collision problems over $f_{N,M}$ does not achieve the optimal time complexity unless $M = N$. The above tradeoff can only achieve $T = \tilde{O}\left(C_k^{1/k} \cdot \frac{M}{N^{1/k}}\right)$, while the optimal complexity should be

$$T_{opt} = \tilde{O}\left(C_k^{1/k} \cdot \frac{M}{M^{1/k}}\right) = \tilde{O}\left(C_k^{1/k} \cdot M^{1-1/k}\right).$$

To achieve the optimal time complexity, we need more memory

$$S = \tilde{O}\left(\frac{C_k^{2/k} \cdot M^{(2k-2)/k}}{N}\right).$$

Since we must create $C_2 = \tilde{\Theta}\left(C_k^{2/k} \cdot M^{(k-2)/k}\right)$ many 2-collisions within T_{opt} .

Time-Space Tradeoff Beyond $C_k^{2/k} N^{(k-2)/k}$. We generalize the above results in the intermediate regime $S \in \left[C_k^{2/k} \cdot N^{(k-2)/k}, \frac{C_k^{2/k} \cdot M^{(2k-2)/k}}{N} \right]$ by deriving another tradeoff by balancing

$$\frac{M}{\sqrt{NS}} \cdot C_2^* \sim \frac{C_3 M}{C_2^*} \sim \dots \sim \frac{C_k M}{C_{k-1}^*},$$

which yields $C_2^* = \tilde{\Theta} \left(C_k^{\frac{1}{k-1}} \cdot (NS)^{\frac{k-2}{2k-2}} \right)$, as long as $C_2^* \lesssim S$, which leads to

$$T = \tilde{O} \left(\frac{C_k^{1/(k-1)} \cdot M}{(NS)^{1/(2(k-1))}} \right), \quad S \in \left[C_k^{2/k} \cdot N^{(k-2)/k}, \frac{C_k^{2/k} \cdot M^{(2k-2)/k}}{N} \right].$$

6.2 Arbitrary Functions

For any function $f_{N,M}$, the decision version of the k -collision problem—also known as *k-element distinctness*—is structurally similar to the 3-element distinctness case. The algorithm proceeds by searching for all 2-collisions and storing them until the available memory is filled. Then, the domain $[N]$ is scanned to identify whether any stored pair extends to a k -collision. Following the analysis for 3-collisions, the time-space tradeoff becomes

$$T = \tilde{O} \left(\frac{N^{3/2}}{\sqrt{S}} + k \cdot \frac{Nn_2}{Sn_k} \right) = \tilde{O} \left(\frac{N^{3/2}}{\sqrt{S}} + \frac{Nn_2}{Sn_k} \right).$$

Similarly, the worst-case tradeoff arises when $n_2 = \Theta(N)$ and $n_k = O(1)$.

In contrast, the *search* problem for arbitrary functions is considerably more complex. The difficulty is that it does not depend solely on n_2 and n_k , but on the distribution of collisions at all intermediate levels. Specifically, the process must iteratively construct C_2 2-collisions, C_3 3-collisions, and so on, until C_k k -collisions are obtained.

Given C_t t -collisions, the time to produce C_{t+1} $(t+1)$ -collisions is: $T_{t+1} = \tilde{O} \left(\frac{C_{t+1}}{C_t} \cdot \frac{n_{c,t} N}{n_{c,t+1}} \right)$.

Therefore, the total time complexity becomes:

$$T = \sum_{t=2}^{k-1} \tilde{O} \left(\frac{C_{t+1}}{C_t} \cdot \frac{n_{c,t} N}{n_{c,t+1}} \right) + \tilde{O} \left(\frac{M\sqrt{S}}{\sqrt{N}} \right).$$

Acknowledgments. The authors would like to thank the shepherd for guidance in improving the rigor of several proofs. We are also grateful to the anonymous reviewers for their detailed and valuable comments. This research was partially supported by the Ministry of Education in Singapore under Grants RG102/24.

References

- [1] Scott Aaronson and Yaoyun Shi. “Quantum lower bounds for the collision and the element distinctness problems”. In: *Journal of the ACM (JACM)* 51.4 (2004), pp. 595–605. DOI: [10.1145/1008731.1008735](https://doi.org/10.1145/1008731.1008735).
- [2] Andris Ambainis. “Quantum Walk Algorithm for Element Distinctness”. In: *SIAM J. Comput.* 37.1 (2007), pp. 210–239. DOI: [10.1137/S0097539705447311](https://doi.org/10.1137/S0097539705447311).
- [3] Richard Arratia, Larry Goldstein, and Louis Gordon. “Two moments suffice for Poisson approximations: the Chen-Stein method”. In: *The Annals of Probability* (1989), pp. 9–25.
- [4] Nikhil Bansal et al. “Faster Space-Efficient Algorithms for Subset Sum, k-Sum, and Related Problems”. In: *SIAM J. Comput.* 47.5 (2018), pp. 1755–1777. DOI: [10.1137/17M1158203](https://doi.org/10.1137/17M1158203).
- [5] Paul Beame, Raphaël Clifford, and Widad Machmouchi. “Element Distinctness, Frequency Moments, and Sliding Windows”. In: *54th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2013, 26-29 October, 2013, Berkeley, CA, USA*. IEEE Computer Society, 2013, pp. 290–299. DOI: [10.1109/FOCS.2013.39](https://doi.org/10.1109/FOCS.2013.39).
- [6] Mihir Bellare and Tadayoshi Kohno. “Hash Function Balance and Its Impact on Birthday Attacks”. In: *Advances in Cryptology – EUROCRYPT 2004*. Ed. by Christian Cachin and Jan Camenisch. Vol. 3027. Lecture Notes in Computer Science. Interlaken, Switzerland: Springer, Berlin, Heidelberg, Germany, May 2004, pp. 401–418. DOI: [10.1007/978-3-540-24676-3_24](https://doi.org/10.1007/978-3-540-24676-3_24).
- [7] Aleksandrs Belovs. “Learning-Graph-Based Quantum Algorithm for k-Distinctness”. In: *53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS 2012, New Brunswick, NJ, USA, October 20-23, 2012*. IEEE Computer Society, 2012, pp. 207–216. DOI: [10.1109/FOCS.2012.18](https://doi.org/10.1109/FOCS.2012.18).
- [8] Aleksandrs Belovs et al. “Time-Efficient Quantum Walks for 3-Distinctness”. In: *Automata, Languages, and Programming - 40th International Colloquium, ICALP 2013, Riga, Latvia, July 8-12, 2013, Proceedings, Part I*. Ed. by Fedor V. Fomin et al. Vol. 7965. Lecture Notes in Computer Science. Springer, 2013, pp. 105–122. DOI: [10.1007/978-3-642-39206-1_10](https://doi.org/10.1007/978-3-642-39206-1_10).
- [9] Huck Bennett, Surendra Ghentiyala, and Noah Stephens-Davidowitz. “The more the merrier! on total coding and lattice problems and the complexity of finding multicollisions”. In: *16th Innovations in Theoretical Computer Science Conference (ITCS 2025)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2025, pp. 14–1. DOI: [10.4230/LIPIcs.ITCS.2025.14](https://doi.org/10.4230/LIPIcs.ITCS.2025.14).
- [10] Xavier Bonnetain et al. “Finding Many Collisions via Reusable Quantum Walks - Application to Lattice Sieving”. In: *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*. Ed. by Carmit Hazay and Martijn Stam.

- Vol. 14008. Lecture Notes in Computer Science. Springer, 2023, pp. 221–251. DOI: [10.1007/978-3-031-30589-4_8](https://doi.org/10.1007/978-3-031-30589-4_8).
- [11] Allan Borodin et al. “A time-space tradeoff for element distinctness”. In: *SIAM Journal on Computing* 16.1 (1987), pp. 97–99. DOI: [10.1137/0216007](https://doi.org/10.1137/0216007).
 - [12] Jan Buzek and Stefano Tessaro. “Collision Resistance from Multi-collision Resistance for All Constant Parameters”. In: *Advances in Cryptology – CRYPTO 2024, Part V*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14924. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2024, pp. 429–458. DOI: [10.1007/978-3-031-68388-6_15](https://doi.org/10.1007/978-3-031-68388-6_15).
 - [13] André Chailloux, María Naya-Plasencia, and André Schrottenloher. “An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography”. In: *Advances in Cryptology – ASIACRYPT 2017, Part II*. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Vol. 10625. Lecture Notes in Computer Science. Hong Kong, China: Springer, Cham, Switzerland, Dec. 2017, pp. 211–240. DOI: [10.1007/978-3-319-70697-9_8](https://doi.org/10.1007/978-3-319-70697-9_8).
 - [14] Lijie Chen et al. “Truly Low-Space Element Distinctness and Subset Sum via Pseudorandom Hash Functions”. In: *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022, Virtual Conference / Alexandria, VA, USA, January 9 - 12, 2022*. Ed. by Joseph (Seffi) Naor and Niv Buchbinder. SIAM, 2022, pp. 1661–1678. DOI: [10.1137/1.9781611977073.67](https://doi.org/10.1137/1.9781611977073.67).
 - [15] Wei Dai, Stefano Tessaro, and Xihu Zhang. “Super-Linear Time-Memory Trade-Offs for Symmetric Encryption”. In: *TCC 2020: 18th Theory of Cryptography Conference, Part III*. Ed. by Rafael Pass and Krzysztof Pietrzak. Vol. 12552. Lecture Notes in Computer Science. Durham, NC, USA: Springer, Cham, Switzerland, Nov. 2020, pp. 335–365. DOI: [10.1007/978-3-030-64381-2_12](https://doi.org/10.1007/978-3-030-64381-2_12).
 - [16] Fraser Daly. *The Stein–Chen Method: Coupling Techniques for Probability Approximations*. https://www.macs.hw.ac.uk/~fd78/Daly_SteinCoupling.pdf. Lecture notes, Heriot–Watt University. Apr. 2020.
 - [17] Itai Dinur. “Tight Time-Space Lower Bounds for Finding Multiple Collision Pairs and Their Applications”. In: *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*. Ed. by Anne Canteaut and Yuval Ishai. Vol. 12105. Lecture Notes in Computer Science. Springer, 2020, pp. 405–434. DOI: [10.1007/978-3-030-45721-1_15](https://doi.org/10.1007/978-3-030-45721-1_15).
 - [18] Itai Dinur et al. “Memory-Efficient Algorithms for Finding Needles in Haystacks”. In: *Advances in Cryptology – CRYPTO 2016, Part II*. Ed. by Matthew Robshaw and Jonathan Katz. Vol. 9815. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Berlin, Heidelberg, Germany, Aug. 2016, pp. 185–206. DOI: [10.1007/978-3-662-53008-5_7](https://doi.org/10.1007/978-3-662-53008-5_7).

- [19] Philippe Flajolet and Andrew M. Odlyzko. “Random Mapping Statistics”. In: *Advances in Cryptology – EUROCRYPT’89*. Ed. by Jean-Jacques Quisquater and Joos Vandewalle. Vol. 434. Lecture Notes in Computer Science. Houthalen, Belgium: Springer, Berlin, Heidelberg, Germany, Apr. 1990, pp. 329–354. DOI: [10.1007/3-540-46885-4_34](https://doi.org/10.1007/3-540-46885-4_34).
- [20] Ashrujit Ghoshal et al. “Hiding in Plain Sight: Memory-Tight Proofs via Randomness Programming”. In: *Advances in Cryptology – EUROCRYPT 2022, Part II*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13276. Lecture Notes in Computer Science. Trondheim, Norway: Springer, Cham, Switzerland, May 2022, pp. 706–735. DOI: [10.1007/978-3-031-07085-3_24](https://doi.org/10.1007/978-3-031-07085-3_24).
- [21] Martin Hellman. “A cryptanalytic time-memory trade-off”. In: *IEEE transactions on Information Theory* 26.4 (2003), pp. 401–406. DOI: [10.1109/TIT.1980.1056220](https://doi.org/10.1109/TIT.1980.1056220).
- [22] Akinori Hosoyamada et al. “Improved Quantum Multicollision-Finding Algorithm”. In: *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019, Chongqing, China, May 8-10, 2019 Revised Selected Papers*. Ed. by Jintai Ding and Rainer Steinwandt. Vol. 11505. Lecture Notes in Computer Science. Springer, 2019, pp. 350–367. DOI: [10.1007/978-3-030-25510-7_19](https://doi.org/10.1007/978-3-030-25510-7_19).
- [23] Joseph Jaeger and Stefano Tessaro. “Tight Time-Memory Trade-Offs for Symmetric Encryption”. In: *Advances in Cryptology – EUROCRYPT 2019, Part I*. Ed. by Yuval Ishai and Vincent Rijmen. Vol. 11476. Lecture Notes in Computer Science. Darmstadt, Germany: Springer, Cham, Switzerland, May 2019, pp. 467–497. DOI: [10.1007/978-3-030-17653-2_16](https://doi.org/10.1007/978-3-030-17653-2_16).
- [24] Siddhartha Jain et al. “On pigeonhole principles and Ramsey in TFNP”. In: *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. 2024, pp. 406–428. DOI: [10.48550/arXiv.2401.12604](https://doi.org/10.48550/arXiv.2401.12604).
- [25] Stacey Jeffery and Sebastian Zur. “Multidimensional quantum walks”. In: *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*. 2023, pp. 1125–1130. DOI: [10.48550/arXiv.2208.13492](https://doi.org/10.48550/arXiv.2208.13492).
- [26] Antoine Joux. “Multicollisions in Iterated Hash Functions. Application to Cascaded Constructions”. In: *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*. Ed. by Matthew K. Franklin. Vol. 3152. Lecture Notes in Computer Science. Springer, 2004, pp. 306–316. DOI: [10.1007/978-3-540-28628-8_19](https://doi.org/10.1007/978-3-540-28628-8_19).
- [27] Antoine Joux and Stefan Lucks. “Improved Generic Algorithms for 3-Collisions”. In: *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*. Ed. by Mitsuru Matsui. Vol. 5912. Lecture Notes in Computer Science. Springer, 2009, pp. 347–363. DOI: [10.1007/978-3-642-10366-7_21](https://doi.org/10.1007/978-3-642-10366-7_21).
- [28] Qipeng Liu and Mark Zhandry. “On Finding Quantum Multi-collisions”. In: *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual Interna-*

- tional Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III*. Ed. by Yuval Ishai and Vincent Rijmen. Vol. 11478. Lecture Notes in Computer Science. Springer, 2019, pp. 189–218. DOI: [10.1007/978-3-030-17659-4_7](https://doi.org/10.1007/978-3-030-17659-4_7).
- [29] Xin Lyu and Weihao Zhu. “Time-Space Tradeoffs for Element Distinctness and Set Intersection via Pseudorandomness”. In: *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*. Ed. by Nikhil Bansal and Viswanath Nagarajan. SIAM, 2023, pp. 5243–5281. DOI: [10.1137/1.9781611977554.CH190](https://doi.org/10.1137/1.9781611977554.CH190).
 - [30] Nikhil S. Mande, Justin Thaler, and Shuchen Zhu. “Improved Approximate Degree Bounds for k-Distinctness”. In: *15th Conference on the Theory of Quantum Computation, Communication and Cryptography, TQC 2020, June 9-12, 2020, Riga, Latvia*. Ed. by Steven T. Flammia. Vol. 158. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, 2:1–2:22. DOI: [10.4230/LIPICS.TQC.2020.2](https://doi.org/10.4230/LIPICS.TQC.2020.2).
 - [31] Mridul Nandi and Douglas R. Stinson. “Multicollision Attacks on Some Generalized Sequential Hash Functions”. In: *IEEE Transactions on Information Theory* 53.2 (2007), pp. 759–767. DOI: [10.1109/TIT.2006.889721](https://doi.org/10.1109/TIT.2006.889721).
 - [32] Paul C. van Oorschot and Michael J. Wiener. “Parallel Collision Search with Cryptanalytic Applications”. In: *J. Cryptol.* 12.1 (1999), pp. 1–28. DOI: [10.1007/PL00003816](https://doi.org/10.1007/PL00003816).
 - [33] John M Pollard. “Monte Carlo methods for index computation (mod p)”. In: *Mathematics of computation* 32.143 (1978), pp. 918–924. DOI: <https://doi.org/10.2307/2006496>.
 - [34] Somindu C. Ramanna and Palash Sarkar. “On Quantifying the Resistance of Concrete Hash Functions to Generic Multicollision Attacks”. In: *IEEE Transactions on Information Theory* 57.7 (2011), pp. 4798–4816. DOI: [10.1109/TIT.2011.2146570](https://doi.org/10.1109/TIT.2011.2146570).
 - [35] Ron D. Rothblum and Prashant Nalini Vasudevan. “Collision-Resistance from Multi-Collision-Resistance”. In: *Advances in Cryptology – CRYPTO 2022, Part III*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13509. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2022, pp. 503–529. DOI: [10.1007/978-3-031-15982-4_17](https://doi.org/10.1007/978-3-031-15982-4_17).
 - [36] Stefano Tessaro and Aishwarya Thiruvengadam. “Provable Time-Memory Trade-Offs: Symmetric Cryptography Against Memory-Bounded Adversaries”. In: *TCC 2018: 16th Theory of Cryptography Conference, Part I*. Ed. by Amos Beimel and Stefan Dziembowski. Vol. 11239. Lecture Notes in Computer Science. Panaji, India: Springer, Cham, Switzerland, Nov. 2018, pp. 3–32. DOI: [10.1007/978-3-030-03807-6_1](https://doi.org/10.1007/978-3-030-03807-6_1).
 - [37] Andrew Chi-Chih Yao. “Near-optimal time-space tradeoff for element distinctness”. In: *SIAM Journal on Computing* 23.5 (1994), pp. 966–975. DOI: [10.1137/S0097539788148959](https://doi.org/10.1137/S0097539788148959).

- [38] Huacheng Yu and Wei Zhan. “Randomized vs. Deterministic Separation in Time-Space Tradeoffs of Multi-Output Functions”. In: *15th Innovations in Theoretical Computer Science Conference (ITCS 2024)*. Ed. by Venkatesan Guruswami. Vol. 287. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 99:1–99:15. ISBN: 978-3-95977-309-6. DOI: [10.4230/LIPIcs.ITCS.2024.99](https://doi.org/10.4230/LIPIcs.ITCS.2024.99).
- [39] Jian Zou, Yongyang Liu, and Le Dong. “An Efficient Quantum Multi-Collision Search Algorithm”. In: *IEEE Access* 8 (2020), pp. 181619–181628. DOI: [10.1109/ACCESS.2020.3028736](https://doi.org/10.1109/ACCESS.2020.3028736).

Appendix A Proofs of Some Statistics in Section 3

Lemma 2. *Under the single- or multi-chain method using a random hash function $H_{M,N}$, for any function $f_{N,M}$ with $M \geq N$ and n_2 2-collisions, constructing s chains of equal length yields an effective maximum chain length*

$$\ell = O\left(\min\left(\sqrt{\frac{N}{s}}, \frac{N}{\sqrt{n_2 s}}\right)\right).$$

In particular, if $f_{N,M}$ is random so that $n_2 = O(N)$, then $\ell = O(\sqrt{N/s})$.

Proof. We first prove the single chain case and then extend to multiple chains.

Single Chain. When constructing the chain, a collision happens if either $x_i \in \{x_1, \dots, x_{i-1}\}$ or $y_i = f(x_i) \in \{y_1, \dots, y_{i-1}\}$.

At each step $i \leq \ell$, the probability that the i -th element avoids collision with the previous $i-1$ elements is

$$\Pr(\text{no collision at step } i) \approx \left(1 - \frac{1}{N} - \frac{n_2}{N(N-1)}\right)^{i-1} \approx \exp\left(-\frac{i-1}{N} - \frac{(i-1)n_2}{N(N-1)}\right).$$

Therefore, the probability of constructing a chain of length ℓ without collision is

$$\prod_{j=1}^{\ell-1} \exp\left(-\frac{j}{N} - \frac{j \cdot n_2}{N(N-1)}\right) \approx \exp\left(-\frac{\ell^2}{2}\left(\frac{1}{N} + \frac{n_2}{N^2}\right)\right).$$

Requiring this to remain a non-negligible constant in $(0,1)$ implies

$$\frac{\ell^2}{2}\left(\frac{1}{N} + \frac{n_2}{N^2}\right) = O(1) \implies \ell = O\left(\min\left(\sqrt{N}, \frac{N}{\sqrt{n_2}}\right)\right).$$

Multiple Chains. When building s chains sequentially, each new point must avoid collisions with all points in the current chain and in the $s-1$ previously constructed chains. The last chain begins with the most previously occupied points (about $(s-1)\ell$), so it has the shortest expected length; bounding its length thus suffices for calculating the upper bound of expected chain length. Concretely, its probability of reaching length ℓ without any collision is

$$\prod_{j=(s-1)\ell}^{s\ell-1} \exp\left(-\frac{j}{N} - \frac{j \cdot n_2}{N(N-1)}\right) = \exp\left(-\frac{n_2}{N^2} \sum_{j=(s-1)\ell}^{s\ell-1} j\right) \approx \exp\left(-\ell^2 s \left(\frac{1}{N} + \frac{n_2}{N^2}\right)\right).$$

Requiring this to remain a non-negligible constant in $(0,1)$ implies

$$\ell^2 s \left(\frac{1}{N} + \frac{n_2}{N^2}\right) = O(1) \implies \ell = O\left(\min\left(\sqrt{\frac{N}{s}}, \frac{N}{\sqrt{n_2 s}}\right)\right).$$

which completes proof. $\square$

Lemma 3. Under the chain-based algorithm method, for any function $f_{N,M} : [N] \rightarrow [M]$, $M \geq N$ with n_2 pairs of 2-collision, given a true random hash function $H_{M,N} : [M] \rightarrow [N]$, constructing $s \ll N$ chains $(x_i^j, y_i^j)_{i \in [\ell], j \in [s]}$ with length $\ell = \min\left(\frac{N}{\sqrt{n_2}s}, \sqrt{\frac{N}{s}}\right)$ by

$$\begin{aligned} x_1^j &\leftarrow \$[N], \quad y_1^j = f_{M,N}(x_1^j), \quad j \in [s] \\ x_i &= H_{M,N}(y_{i-1}^j), \quad y_i^j = f_{M,N}(x_i^j), \quad i \in [2, \ell], \quad j \in [s] \end{aligned}$$

Denote the number of distinct points x_i^j over the chains as n_d , then

$$\Pr(n_d \geq \frac{1}{4} s \ell) \geq 1 - \exp(-\Omega(s)).$$

Proof. We start with the single chain case then go to the multi-chains case which is much more complicated.

Single chain. Let T be the index of the first collision—either

$$x_i \in \{x_1, \dots, x_{i-1}\} \quad \text{or} \quad y_i = f(x_i) \in \{y_1, \dots, y_{i-1}\},$$

capped at ℓ . At each step $i \leq \ell$, the collision probability satisfies

$$\Pr(\text{collision at step } i) \leq \frac{i-1}{N} + \frac{(i-1)n_2}{N(N-1)} \leq \frac{\ell}{N} + \frac{\ell n_2}{N^2}.$$

Hence the expected time to the first collision is

$$\mathbb{E}[T] \geq \frac{1}{\frac{\ell}{N} + \frac{\ell n_2}{N^2}} = \frac{N^2}{\ell(N + n_2)}.$$

$$\text{If } n_2 \leq N, \ell = \sqrt{N} \implies \mathbb{E}[T] \geq \frac{1}{2} \sqrt{N} = \frac{1}{2} \ell,$$

$$\text{If } n_2 \geq N, \ell = \frac{N}{\sqrt{n_2}} \implies \mathbb{E}[T] \geq \frac{1}{2} \frac{N}{\sqrt{n_2}} = \frac{1}{2} \ell.$$

Since $n_d = T$ (we stop at the first collision or at ℓ), it follows that

$$\mathbb{E}[n_d] \geq \frac{1}{2} \ell.$$

Define $X = \ell - n_d$, so that $\mathbb{E}[X] = \ell - \mathbb{E}[n_d] \leq \frac{1}{2} \ell$. By Markov's inequality, for any $a > 0$,

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

Choose $a = \frac{3}{4} \ell$. Then

$$\Pr(X \geq \frac{3}{4} \ell) \leq \frac{\frac{1}{2} \ell}{\frac{3}{4} \ell} = \frac{2}{3}.$$

Therefore

$$\Pr(n_d > \frac{1}{4} \ell) = 1 - \Pr(X \geq \frac{3}{4} \ell) \geq 1 - \frac{2}{3} = \frac{1}{3}.$$

Multi-chains. We intend to lower bound $\Pr(n_d^N \geq t)$ by creating some new chains $\{x_i^{j'}, y_i^{j'}\}$ based on $\{x_i^j, y_i^j\}$ and denote the number of distinct points as n_d' , the generation method is shown below

- For j -th new chain, we firstly compute the number $n_d^{<j'}$ points (already distinct) for all previous $j-1$ chains, and we sample $(j-1)\ell - n_d^{<j'}$ distinct points that are distinct for points in all previous chains. Now we have $(j-1)\ell$ distinct points. (Note that $\ell = \min\left(\frac{N}{\sqrt{n_2 s}}, \sqrt{\frac{N}{s}}\right)$).
- Then we generate j -th chain by letting $(x_i^{j'} = x_i^j, y_i^{j'} = y_i^j)$ starting from $i = 1$ until we get a self collision or get a collision with previous $(j-1)\ell$ distinct points. After that, we stop the generation and only keep the distinct points. At the end we also need to remove the $(j-1)\ell - n_d^{<j'}$ sampled from above step so we can move to next $(j+1)$ -th chain.

Then we can observe that $\forall x, \Pr\left(x \in \{x_i^{j'}\}\right) \leq \Pr\left(x \in \{x_i^j\}\right)$, which means that

$$\Pr(n_d \geq t) \geq \Pr(n_d' \geq t), \quad \forall t \geq 0$$

In addition, since $\{x_i^j, y_i^j\}$ are random, so the generation of $\{x_i^j, y_i^j\}$ is equivalent to first sampling $(j-1)\ell$ distinct points then create the j -th chain until it hits some random points in previous $(j-1)\ell$ points. As a result, the number of distinct points over the j -th chain is independent from all other chains.

Since each chain can be analyzed separately, we compute the expected number of distinct points per chain. Let T_s be the index of the first collision in the last chain—either x_i or $y_i = f(x_i)$, capped at ℓ . For each step $i \leq \ell$, define

$$\begin{aligned} p_i &= \Pr(T_s = i \mid T_s \geq i) \\ &\leq \frac{i-1 + (s-1)\ell}{N} + \frac{(i-1 + (s-1)\ell)n_2}{N(N-1)} \\ &\leq \frac{s\ell}{N} + \frac{s\ell n_2}{N(N-1)} =: p. \end{aligned}$$

Then $\Pr(T_s > t) = \prod_{i=1}^t (1 - p_i) \geq (1 - p)^t$, with $\Pr(T_s > 0) = 1$. Thus the expectation can be bounded as

$$\begin{aligned} \mathbb{E}[T_s] &= \sum_{t \geq 0} \Pr(T_s > t) \geq \sum_{t \geq 0} (1 - p)^t = \frac{1}{p} \\ &= \frac{1}{\frac{s\ell}{N} + \frac{s\ell n_2}{N(N-1)}} = \frac{N(N-1)}{s\ell(N + n_2 - 1)} \simeq \frac{N^2}{s\ell(N + n_2 - 1)}. \end{aligned}$$

In particular:

$$\begin{aligned} \text{If } n_2 \leq N, \quad \ell = \sqrt{\frac{N}{s}} &\implies \mathbb{E}[T_s] \geq \frac{1}{2} \sqrt{\frac{N}{s}} = \frac{\ell}{2}, \\ \text{If } n_2 \geq N, \quad \ell = \frac{N}{\sqrt{n_2 s}} &\implies \mathbb{E}[T_s] \geq \frac{1}{2} \frac{N}{\sqrt{n_2 s}} = \frac{\ell}{2}. \end{aligned}$$

Since we have s chains, let T_i denote the first-collision index for the i -th chain. By the same analysis,

$$\ell \geq \mathbb{E}[T_i] \geq \mathbb{E}[T_s] \geq \frac{\ell}{2}, \quad \forall i \leq s.$$

Moreover, in the decoupled model the variables $\{T_i\}$ are independent and bounded: $1 \leq T_i \leq \ell$, and we also have $n'_d := \sum_{i=1}^s T_i - s$. Then

$$\frac{1}{2} s \ell - s \leq \mathbb{E}[n'_d] \leq s \ell - s.$$

Applying Hoeffding's inequality to the independent bounded variables $T_i \in [0, \ell]$, for any $c \in (0, 1)$ we have

$$\begin{aligned} \Pr(|n'_d - \mathbb{E}[n'_d]| \geq (1-c) \frac{1}{2} s \ell - s) \\ &= \Pr\left(\left|\sum_{i=1}^s T_i - \sum_{i=1}^s \mathbb{E}[T_i]\right| \geq s \left(\frac{1-c}{2} \ell - 1\right)\right) \\ &\leq 2 \exp\left(-\frac{2s^2 \left(\frac{1-c}{2} \ell - 1\right)^2}{\sum_{i=1}^s (\ell - 1)^2}\right) \\ &= 2 \exp\left(-\frac{2s \left(\frac{1-c}{2} \ell - 1\right)^2}{(\ell - 1)^2}\right) = \exp(-\Omega(s)). \end{aligned}$$

Using $\mathbb{E}[n'_d] \geq \frac{1}{2} s \ell - s$ and setting $c = \frac{1}{2}$ (requiring $\ell > 4$ for a positive deviation), we obtain

$$\begin{aligned} \Pr(n'_d \geq \tfrac{1}{4} s \ell) &= 1 - \Pr(n'_d < \tfrac{1}{4} s \ell) \\ &= 1 - \Pr(n'_d - \mathbb{E}[n'_d] < \tfrac{1}{4} s \ell - \mathbb{E}[n'_d]) \\ &\geq 1 - \Pr(|n'_d - \mathbb{E}[n'_d]| \geq \mathbb{E}[n'_d] - \tfrac{1}{4} s \ell) \\ &\geq 1 - \Pr(|n'_d - \mathbb{E}[n'_d]| \geq \tfrac{1}{2} s \ell - s - \tfrac{1}{4} s \ell) \\ &= 1 - \Pr(|n'_d - \mathbb{E}[n'_d]| \geq s \left(\tfrac{1}{4} \ell - 1\right)) \\ &\geq 1 - \exp(-\Omega(s)). \end{aligned}$$

This completes the proof. $\square$

Lemma 4. Let $f_{N,M} : [N] \rightarrow [M]$ be a random function, and let n_2 denote the number of collisions of $f_{N,M}$. If $M \leq N^2$, then

$$\Pr\left(n_2 \geq \frac{N(N-1)}{4M}\right) \geq 1 - 2 \exp(-\Omega(N^2/M)).$$

Moreover, the event $n_2 \leq O(N)$ occurs except for negligible probability for all $M \geq N$. More precisely, for the upper tail, we have $\Pr(n_2 \leq \frac{3N(N-1)}{2M}) \geq 1 - \frac{4}{\sqrt{N}}$ for all $M \ll N^2$. In particular, when $M \sim N^2$, there exists a constant $c > 0$ such that $\Pr(n_2 \leq \frac{cN(N-1)}{2M}) \geq 1 - \frac{4}{N}$.

Proof. For each pair $1 \leq i < j \leq N$ define the indicator

$$X_{i,j} = \mathbf{1}\{f(i) = f(j)\}, \quad n_2 = \sum_{i < j} X_{i,j}.$$

Then

$$\mu = \mathbb{E}[n_2] = \sum_{i < j} \Pr(f(i) = f(j)) = \frac{N(N-1)}{2M}.$$

Two variables $X_{i,j}$ and $X_{k,\ell}$ are dependent only if $\{i, j\} \cap \{k, \ell\} \neq \emptyset$. A straightforward count shows

$$\Delta = \sum_{(i,j) \sim (k,\ell)} \mathbb{E}[X_{i,j} X_{k,\ell}] = 2 \sum_{(i < j) \sim (k < \ell)} \mathbb{E}[X_{i,j} X_{k,\ell}] = \frac{N(N-1)(N-2)}{M^2}.$$

By Janson's inequality, for any $0 < \epsilon < 1$,

$$\Pr(n_2 \leq (1 - \epsilon)\mu) \leq \exp\left(-\frac{\epsilon^2 \mu^2}{2(\mu + \Delta)}\right)$$

Taking $\epsilon = \frac{1}{2}$ and noting $\Delta = O(\mu)$ for $M \leq N^2$ gives both tails bounded by $\exp(-\Omega(\frac{N^2}{M}))$. Since

$$\left(1 - \frac{1}{2}\right) \mu = \frac{N(N-1)}{4M}$$

then we have proved the lower tail.

For the upper tail, in general, it is hard to get a *uniform* exponentially small bound in μ . The upper tail is usually difficult in probability. Instead, we aim to *upper bound* the probability by a quantity that is negligible in the cryptographic sense. We divide into two cases. When $M \geq N^{3/2}$, the dependence among the indicators $X_{i,j}$ is small; we measure the distance of n_2 from a Poisson distribution via the total-variation parameter d_{TV} . When $M \leq N^{3/2}$, we use a simpler method, Chebyshev, which already yields a negligible bound since μ is large.

(-) $M \leq N^{3/2}$. We first compute the variance:

$$\begin{aligned}
\mathbb{V}ar(n_2) &= \mathbb{E}[n_2^2] - \mu^2 \\
&= \mathbb{E}\left[\left(\sum_{(i,j)} X_{i,j}\right)^2\right] - \mu^2 \\
&= \mathbb{E}\left[\sum_{(i,j)} (X_{i,j})^2\right] + \mathbb{E}\left[\sum_{(i,j) \sim (k,\ell)} (X_{i,j} X_{k,\ell})\right] + \mathbb{E}\left[\sum_{(i,j) \cap (k,\ell) = \emptyset} (X_{i,j} X_{k,\ell})\right] - \mu^2 \\
&= \mu + \Delta + \frac{N(N-1)(N-2)(N-3)}{4M^2} - \mu^2 \\
&= \frac{N(N-1)}{2M} - \frac{N(N-1)}{2M^2} \sim \frac{N^2}{2M}
\end{aligned}$$

Then by Chebyshev,

$$\Pr(n_2 \geq c\mu) \leq \frac{\mathbb{V}ar(n_2)}{(c-1)^2\mu^2} \sim \frac{1}{(c-1)^2} \cdot \frac{2M}{N^2} \leq \frac{2}{(c-1)^2\sqrt{N}}.$$

In particular, for $c = 3$ and $M \leq N^{3/2}$, $\Pr(n_2 \geq 3\mu) \leq \frac{1}{2\sqrt{N}}$, which is negligible as N grows.

(-) $M \geq N^{3/2}$. We upper bound the tail via the Chen–Stein method for the birthday problem (see [3, 16]). Let $X_p \sim \text{Poisson}(\lambda)$ with $\lambda = \mu$. Then the total-variation distance satisfies

$$d_{\text{TV}}(n_2, X_p) \leq \frac{8\lambda(1 - e^{-\lambda})}{N} \leq \frac{8}{N} \cdot \frac{N(N-1)}{2M} \leq \frac{4N}{M}.$$

By the definition of total variation, for every threshold c ,

$$\Pr(n_2 \geq c\mu) \leq \Pr(X_p \geq c\mu) + d_{\text{TV}}(n_2, X_p).$$

For the Poisson tail, Chernoff's bound gives

$$\Pr(X_p \geq c\mu) \leq \exp(-\mu \Omega(c)).$$

Hence, when $N^{3/2} \leq M \ll N^2$ we have $\mu \rightarrow \infty$, so $\Pr(X_p \geq 3\mu)$ is exponentially small in μ and

$$\Pr(n_2 \geq 3\mu) \leq d_{\text{TV}}(n_2, X_p) + \Pr(X_p \geq 3\mu) \lesssim \frac{4N}{M} \leq \frac{4}{\sqrt{N}}.$$

When $M \sim N^2$, we have $\mu \sim \frac{1}{2}$. In this case the Poisson tail decays exponentially in c , so by choosing c sufficiently large we can make the probability negligible compared to $4/N$.

As a result, for any $N^{3/2} \leq M \ll N^2$,

$$\Pr(n_2 \geq 3\mu) \leq d_{\text{TV}}(n_2, X_p) + \Pr(X_p \geq 3\mu) \lesssim \frac{4N}{M} \leq \frac{4}{\sqrt{N}},$$

and when $M \sim N^2$, choosing c as above yields

$$\Pr(n_2 \geq c\mu) \leq d_{TV}(n_2, X_p) + \Pr(X_p \geq c\mu) \lesssim \frac{4N}{M} = \frac{4}{N}.$$

In addition, for $M \geq N$ we have $\mu = \Theta(\frac{N^2}{M}) \leq \frac{N}{2}$. Therefore, for any threshold of order $\Omega(N)$ we obtain

$$\Pr(n_2 \geq \Omega(N))$$

is negligible, as follows directly from the above tail bounds. $\square$

Appendix B Proof for Lemma 5

1. *Variant of Markov's inequality:*

$$\Pr(X \geq a) \geq \frac{\mathbb{E}[X] - a}{\mathbb{E}[X \mid X \geq a] - a}.$$

Proof. We have

$$\begin{aligned} \mathbb{E}[X] &= \Pr(X \geq a) \mathbb{E}[X \mid X \geq a] + \Pr(X < a) \mathbb{E}[X \mid X < a] \\ &\leq \Pr(X \geq a) \mathbb{E}[X \mid X \geq a] + (1 - \Pr(X \geq a)) a \\ &= \Pr(X \geq a) (\mathbb{E}[X \mid X \geq a] - a) + a. \end{aligned}$$

Rearranging gives

$$\Pr(X \geq a) \geq \frac{\mathbb{E}[X] - a}{\mathbb{E}[X \mid X \geq a] - a}.$$

Lemma 5. *Given oracle access to any function $f_{N,M}$, $M \geq N$ with n_2 2-collisions and a random hash function $H_{M,N}$, there is an algorithm $\mathcal{A}_{base}^2$ runs in time $O\left(\sqrt{\frac{S}{\log M}} \min\left(\sqrt{N}, \frac{N}{\sqrt{n_2}}\right)\right)$ using space S for any $S \geq \Omega(\log M)$ bits, such that for every pair of $x_1 \neq x_2 \in [N]$ with $f_{N,M}(x_1) = f_{N,M}(x_2)$,*

$$\Pr[(x_1, x_2) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N}, S)] \geq \Omega\left(\frac{S}{\log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right)\right),$$

In particular, if we have $a = \frac{S \min(\frac{n_2}{N}, 1)}{2 \log M} \geq 1$, then the probability of outputting $\Theta(a)$ different 2-collisions in one run is $\Omega(1)$. If we have $a < 1$, then the probability of outputting at least one 2-collision in $\frac{1}{a}$ independent runs is $\Omega(1)$.

Proof. The algorithm $\mathcal{A}_{base}^2$ proceeds as follows:

1. If $S \sim \log M$, set $s = 1$ and $\ell = \min\left(\frac{N}{\sqrt{n_2}}, \sqrt{N}\right)$. Otherwise, compute $s = \lceil \frac{S}{2 \log M} \rceil$ and set

$$\ell = \min\left(\sqrt{\frac{N}{s}}, \frac{N}{\sqrt{n_2 \cdot s}}\right).$$

2. Run $\mathcal{A}_{chain}(\ell, s, f_{N,M}, H_{M,N})$ and output whatever is returned by $\mathcal{A}_{chain}$.

Therefore, the time-space tradeoff of $\mathcal{A}_{base}^2$ matches that of $\mathcal{A}_{chain}$, which is

$$T = O(s\ell) = O\left(\min\left(\sqrt{Ns}, \frac{N\sqrt{s}}{\sqrt{n_2}}\right)\right) = O\left(\sqrt{\frac{S}{\log M}} \min\left(\sqrt{N}, \frac{N}{\sqrt{n_2}}\right)\right).$$

Since the chain elements are outputs from a random function, the set of distinct points visited in one run is uniformly distributed over $[N]$, regardless of whether they originate from a single chain or from multiple chains. Therefore, the probability of detecting a given collision can be bounded in terms of the number of distinct random points. Specifically, for a fixed collision pair (x_1, x_2) , let

$$E = \left\{ \exists x_{i_1}^{j_1}, x_{i_2}^{j_2} \text{ on some chains such that } x_1 = x_{i_1}^{j_1}, x_2 = x_{i_2}^{j_2} \right\},$$

we have

$$\begin{aligned} \Pr[(x_1, x_2) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N})] &= \Pr[E] \\ &\geq \Pr[E \mid n_d \geq \frac{s\ell}{4}] \Pr(n_d \geq \frac{s\ell}{4}) \sim \left(\frac{s\ell/4}{N}\right)^2 (1 - \exp(-\Omega(s))) \\ &\geq \frac{S}{32 \log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right) = \Omega\left(\frac{S}{\log M} \min\left(\frac{1}{N}, \frac{1}{n_2}\right)\right). \end{aligned}$$

For the upper bound, since at most $s\ell$ distinct points are examined, we have for any fixed (x_1, x_2)

$$\Pr[(x_1, x_2) \in \mathcal{A}_{base}^2(f_{N,M}, H_{M,N})] = \frac{\binom{D}{2}}{\binom{N}{2}} \leq \frac{\binom{s\ell}{2}}{\binom{N}{2}} \leq \frac{s\ell(s\ell-1)}{N(N-1)} \lesssim \left(\frac{s\ell}{N}\right)^2,$$

where D is the number of distinct points visited.

Let X be the number of 2-collisions output in a single run, and let X_t be the total over t independent runs. Conditioning on the high-probability event $n_d \geq s\ell/4$, we have

$$\frac{1}{32}a \leq \mathbb{E}[X \mid n_d \geq s\ell/4] \Pr(n_d \geq s\ell/4) \leq \mathbb{E}[X] \lesssim n_2 \left(\frac{s\ell}{N}\right)^2 = a.$$

To complete the proof of the lemma on the number of collisions output by $\mathcal{A}_{base}^2$, we distinguish two regimes according to the parameter a .

(i) *Dense regime* ($a \geq 1$). In this regime, a single run outputs $\mathbb{E}[X] = \Theta(1)$ collisions on average. Our goal is to bound the probability that the output size is of the expected order, namely

$$\Pr(X = \Theta(a)).$$

(ii) *Sparse regime* ($a < 1$). Here, the expected number of collisions per run is less than one, so it takes roughly $1/\mathbb{E}[X]$ runs to obtain a collision on average. Setting $t = 1/a$ and running independently, we have

$$\mathbb{E}[X_t] = t \mathbb{E}[X] = \Theta(1),$$

and we aim to bound

$$\Pr(X_t = \Theta(1)).$$

In both regimes, bounding the upper tail of X (or X_t) is important: it controls the probability of producing an excessively large number of collisions, which is undesirable when $\mathcal{A}_{\text{base}}^2$ is used as a subroutine for higher-order collision finding (e.g., 3-collisions or k -collisions).

Furthermore, the two regimes can also be characterized in terms of the number of 2-collisions $n_{c,2}$. Specifically:

$$a \geq 1 \iff n_{c,2} \geq \frac{N^2}{(s\ell)^2}, \quad a < 1 \iff n_{c,2} < \frac{N^2}{(s\ell)^2}.$$

This equivalence will be useful for analyzing the dependence between collisions in the two regimes.

Upper tail. We first handle the upper tail, which is straightforward if we only require a constant-probability bound. By Markov's inequality, for any $C > 0$:

$$\Pr(X \geq C \cdot a) \leq \frac{\mathbb{E}[X]}{C \cdot a} \leq \frac{1}{C}.$$

Similarly, in the sparse regime with $t = 1/a$,

$$\Pr(X_t \geq C) \leq \frac{\mathbb{E}[X_t]}{C} = \frac{t \mathbb{E}[X]}{C} \leq \frac{1}{C}.$$

Two-sided bound. If we can also establish a constant-probability *lower tail* bound, namely that there exist constants $c, c' > 0$ such that

$$\Pr(X \geq c \cdot a) \geq c' \quad \text{or} \quad \Pr(X_t > 0) \geq c',$$

then choosing C sufficiently large gives

$$\Pr(ca \leq X \leq Ca) \geq c' - \frac{1}{C} = \Omega(1),$$

and analogously for X_t in the sparse regime.

Lower tail. We now compute the lower tail, choosing to directly evaluate the dependence between collisions.

(-) *The simple case: very dense regime $a \geq \Omega(s)$.* Before tackling the harder cases, we first note that in the *super-dense* setting, namely $n_{c,2} \geq \Omega(N)$, the proof becomes much simpler. Recall that

$$a = \frac{(s\ell)^2 n_2}{N^2} = \frac{\left(\frac{sN}{\sqrt{n_2 s}}\right)^2 n_2}{N^2} = s.$$

Furthermore, for $s \gg 1$ we have

$$\mathbb{E}[X] \geq \frac{n_2(s\ell)^2}{16N^2} (1 - e^{-\Omega(s)}) \geq \frac{1}{32} a.$$

Since each chain can output at most two collisions, we can also bound

$$\mathbb{E}[X \mid X \geq a] \leq 2s.$$

Applying the variant of Markov's inequality and choosing $c = \frac{1}{64}$, we obtain

$$\Pr(X \geq ca) \geq \frac{\mathbb{E}[X] - ca}{\mathbb{E}[X \mid X \geq ca] - ca} \geq \frac{\frac{1}{32}s - cs}{2s - cs} \geq \frac{1}{128} = \Omega(1).$$

(-) *Dense regime* $a \geq 1$. Let

$$C_2 = \{(i, j) \in [N]^{(2)} : i \neq j, f(i) = f(j)\}, \quad |C_2| = n_{c,2}.$$

In one run, let $D \subseteq [N]$ be the set of distinct points visited, and let $d_0 := s\ell/4$. From lemma 3,

$$\Pr(n_d = |D| \geq d_0) = 1 - e^{-\Omega(s)}.$$

For each collision $(i, j) \in C_2$ define the indicator

$$X_{i,j} := \mathbf{1}\{i \in D \text{ and } j \in D\}.$$

The total number of 2-collisions output is

$$X = \sum_{(i,j) \in C_2} X_{i,j}.$$

Condition on the event $E : \{n_d \geq d_0\}$, and define a random d_0 -subset of D , denoted as D_{d_0} , we can let

$$X_{d_0} := \sum_{(i,j) \in C_2} \mathbf{1}\{i \in D_{d_0} \text{ and } j \in D_{d_0}\}.$$

Since $X \geq X_d$ almost surely, for any a we have

$$\{X \geq a \mid n_d \geq d_0\} \supseteq \{X_d \geq a \mid n_d \geq d_0\}$$

which means

$$\Pr(X \geq a \mid n_d \geq d_0) \geq \Pr(X_{d_0} \geq a \mid n_d \geq d_0).$$

For any fixed (i, j) ,

$$p_2 := \Pr(i, j \in D_{d_0}) = \frac{\binom{d_0}{2}}{\binom{N}{2}} \simeq \frac{d_0^2}{N^2}.$$

Thus we can define

$$\mu := \mathbb{E}[X_{d_0}] = n_{c,2} p_2 \simeq n_{c,2} \frac{(s\ell/4)^2}{N^2} = \Theta(a).$$

We observe that all collisions among the $n_d = \frac{1}{4}s\ell$ sampled points are not independent, but the dependencies fall into two distinct types:

- Type $\sim_1$ (disjoint collisions): $X_{i,j} \sim_1 X_{k,\ell}$ if and only if $\{i, j\} \cap \{k, \ell\} = \emptyset$.
- Type $\sim_2$ (one shared point): $X_{i,j} \sim_2 X_{j,k}$ if the two collisions share exactly one vertex.

Then we can compute

$$\begin{aligned}
\Delta &= \sum_{X_{i,j} \sim X_{k,\ell}} \mathbb{E}[X_{i,j} X_{k,\ell}] \\
&= \sum_{X_{i,j} \sim_1 X_{k,\ell}} \mathbb{E}[X_{i,j} X_{k,\ell}] + \sum_{X_{i,j} \sim_2 X_{j,k}} \mathbb{E}[X_{i,j} X_{j,k}] \\
&\leq \sum_{X_{i,j} \sim_1 X_{k,\ell}} \Pr(X_{i,j} = 1) \Pr(X_{k,\ell} = 1) + \sum_{X_{i,j} \sim_2 X_{j,k}} \mathbb{E}[X_{i,j} X_{j,k}] \\
&\leq \mu^2 + \sum_{X_{i,j} \sim_2 X_{j,k}} \mathbb{E}[X_{i,j} X_{j,k}],
\end{aligned}$$

If two collisions share exactly one vertex, say (i, j) and (i, k) , then

$$p_3 := \Pr(i, j, k \in D_{d_0}) = \frac{\binom{d_0}{3}}{\binom{N}{3}} \simeq \frac{d_0^3}{N^3}, \quad \frac{p_3}{p_2} \simeq \frac{d_0}{N},$$

Given n_2 collisions, there are at most $2n_2$ vertices participating in collisions. For a collision vertex v , let $\deg(v)$ denote the number of collisions (u, v) it participates in. Then

$$\sum_{X_{i,j} \sim_2 X_{j,k}} \mathbb{E}[X_{i,j} X_{j,k}] \leq \sum_{v \text{ collision vertex}} \binom{\deg(v)}{2} p_3.$$

This gives

$$\begin{aligned}
\sum_{X_{i,j} \sim_2 X_{j,k}} \mathbb{E}[X_{i,j} X_{j,k}] &\leq \sum_{v_c} \frac{1}{2} \deg(v_c) (\deg(v_c) - 1) \frac{(s\ell/4)^3}{N^3} \\
&\leq \sum_{v_c} \frac{1}{2} \deg(v_c) (\Delta^* - 1) \frac{(s\ell/4)^3}{N^3} \\
&= n_2 (\Delta^* - 1) \frac{(s\ell/4)^3}{N^3},
\end{aligned}$$

where $\Delta^* := \max_{v_c} \deg(v_c) \leq 2\sqrt{n_2}$, since if some v_c had $\deg(v_c) \geq 2\sqrt{n_2}$ then the number of collisions would exceed $n_{c,2}$.

From earlier we have $\mathbb{E}[X_{d_0} \mid n_d \geq s\ell/4] \geq \Omega(a)$, so there exists a constant $c > 0$ such that

$$ca \leq (1 - \varepsilon)\mu, \quad \text{where} \quad \mu = \mathbb{E}[X_{d_0} \mid n_d \geq s\ell/4].$$

Applying Janson's inequality, for any $0 < \varepsilon < 1$,

$$\begin{aligned} \Pr(X_{d_0} \leq ca \mid n_d \geq s\ell/4) &\leq \Pr(X_{d_0} \leq (1 - \varepsilon)\mu \mid n_d \geq s\ell/4) \\ &\leq \exp\left(-\frac{\varepsilon^2 \mu^2}{2(\mu + \Delta)}\right) \\ &\leq \exp\left(-\frac{\varepsilon^2 \mu^2}{2\left(\mu + \mu^2 + n_2(\Delta^* - 1)\frac{(s\ell)^3}{(4N)^3}\right)}\right). \end{aligned}$$

When $a \geq 1$, we have $\mu = \Omega(1)$, so μ^2 dominates μ which gives us

$$\begin{aligned} \Pr(X_{d_0} \leq ca \mid n_d \geq s\ell/4) &\leq \exp\left(-c' \frac{\mu^2}{\mu^2 + n_2(\Delta^* - 1)\frac{s\ell^3}{(4N)^3}}\right) \\ &\leq \exp\left(-c' \frac{(1+c)^2 a^2}{(1+c)^2 a^2 + n_2(\Delta^* - 1)\frac{(s\ell)^3}{(4N)^3}}\right) \\ &= \exp\left(-c'' \frac{1}{1 + \frac{\Delta^* - 1}{n_2} \cdot \frac{4N}{(1+c)^2 s\ell}}\right). \end{aligned}$$

This is $\Omega(1)$ as long as $\frac{N}{n_2 s\ell}(\Delta^* - 1) = O(1)$. Indeed,

$$\frac{N}{n_2 s\ell}(\Delta^* - 1) \leq \frac{N}{n_2 s\ell} \cdot O(\sqrt{n_2}) = O\left(\frac{N}{s\ell}\right) = O(1),$$

which holds whenever $a \geq 1$ since this implies $n_{c,2} \geq \frac{N^2}{(s\ell)^2}$.

Therefore, for $a \geq \Omega(1)$,

$$\Pr(X \geq a) \geq \Pr(X \geq a \mid n_d \geq s\ell/4) \geq \Pr(X_{d_0} \geq a \mid n_d \geq s\ell/4) = \Omega(1).$$

(-) *Sparse regime* $a < O(1)$. Now we study the X_t for $t = \frac{1}{a}$, where X_t denotes the number of collisions output by t independent runs of $\mathcal{A}_{\text{base}}^2$. From our previous analysis, we know that

$$n_2 < O\left(\frac{N^2}{(s\ell)^2}\right) < N.$$

We can bound the lower tail as follows:

$$\Pr(X_t \geq 1) \geq 1 - \Pr(X < 1)^t.$$

To estimate $\Pr(X < 1)$, we proceed similarly to the earlier case by evaluating dependencies. Conditioning on $n_d \geq \frac{1}{4}s\ell$, we have

$$\Pr(X_{d_0} < 1 \mid n_d \geq \frac{1}{4}s\ell) = \Pr(X_{d_0} = 0 \mid n_d \geq \frac{1}{4}s\ell).$$

Using the zero-event form of Janson's inequality,

$$\Pr(X_{d_0} = 0 \mid n_d \geq \frac{1}{4}s\ell) \leq \exp\left(-\frac{\mu^2}{2(\mu + \Delta)}\right),$$

where $\mu = E[X_d \mid n_d \geq \frac{1}{4}s\ell]$ and Δ is defined as before. Since Δ and μ have the same forms as in the previous computation, we get

$$\begin{aligned} \Pr(X_{d_0} = 0 \mid n_d \geq \frac{1}{4}s\ell) &\leq \exp\left(-\frac{\mu^2}{2\left(\mu + \mu^2 + \frac{(\Delta^*-1)n_2(s\ell)^3}{(4N)^3}\right)}\right) \\ &\leq \exp\left(-\frac{\mu^2}{2\left(2\mu + \frac{(\Delta^*-1)n_2(s\ell)^3}{(4N)^3}\right)}\right) \\ &= \exp\left(-\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}\right), \end{aligned}$$

where we used $\mu = O(1)$ in this regime so μ dominates μ^2

Thus,

$$\Pr(X_{d_0} \geq 1 \mid n_d \geq \frac{1}{4}s\ell) \geq 1 - \exp\left(-\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}\right).$$

Multiplying by the probability of the conditioning event (which is constant when $s \gg 1$), we have

$$\Pr(X_{d_0} \geq 1) \geq 1 - \exp\left(-\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}\right).$$

Now, for t runs,

$$\Pr(X_t \geq 1) \geq 1 - \left[\exp\left(-\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}\right)\right]^t.$$

We require

$$\left[\exp\left(-\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}\right)\right]^t = O(1),$$

which holds provided

$$\frac{\mu}{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}} \cdot t = \Omega(1).$$

This gives the condition

$$t \geq \Omega\left(\frac{4 + c \cdot \frac{(\Delta^*-1)(s\ell)}{N}}{\mu}\right).$$

Since $a \geq \mu$ and

$$\Delta^* - 1 = \sqrt{2n_2}, \quad \text{with } n_2 < \frac{N^2}{(s\ell)^2} \iff \frac{c(\Delta^* - 1)(s\ell)}{N} < 2c,$$

we finally have

$$t = \frac{1}{a} = \Omega\left(\frac{4 + c \cdot \frac{(\Delta^* - 1)(s\ell)}{N}}{\mu}\right),$$

which completes the proof. □